Department of Mechanical Engineering
FACULTY OF ENGINEERING AND DESIGN

FINAL YEAR MEng PROJECT REPORT

Predictive Control of Autonomous Quadcopters

Jean-Michel Alvarez

Submission Date: May 9, 2022

Word count: 9396

I certify that I have read and understood the entry in the Student Handbook for the Department of Mechanical Engineering on Cheating and Plagiarism and that all material in this assignment is my own work, except where I have indicated with appropriate references.

Jean-Michel Alvarez

Supervisor: Dr. Andrew Hillis

Assessor: Prof. Gary Lock

Abstract

As quadcopters become increasingly present in urban environments, the level of safety of UAS demanded by the public equally increases. The development of a robust Multiple-MPC controller is detailed in this report. Many articles have been published on the subject of robust quadcopter control, discussing the merits of various control methods such as cascaded PID, LQR, MRAC, and hybrid MPC, among others. The mathematical model of the quadcopter, based on a DJI F450, is obtained from a previous research paper written by a former lecturer at the University of Bath, Dr. Jon du Bois. Research is primarily done in the MATLAB Simulink programming block environment in direct relation to du Bois' quadcopter model. Various aspects of LQR and MPC control methods are analysed. The voltage saturation of the LQR controller is compared to the direct implementation of physical constraints into the MPC controller. The latter method is found to have better performance in both vertical and horizontal flight. Rotor failure scenarios are then evaluated, with LQR and single MPC displaying unstable performance whereas the Multiple MPC controller is able to follow the reference trajectory with minimal disturbance. However, the complexity of NLMPC simulations was limited by exponential increases in computational time, with the sample time having to be increased to obtain reliable optimisation results. Measures were taken to mitigate this cost, such as supplying initial guesses to the solver, limiting the number of iterations per timestep, and enabling suboptimal solutions, which finally allowed for the obtaining of realistic results. Future work could be conducted in 1) the fields of sensor suite integration, allowing for sensor noise, 2) gain-scheduled LQR, a hybrid control method that borrows from both multiple MPC and LQR, and 3) the addition of various external disturbances to the system.

Acknowledgements

On an academic note, I would like to thank Dr. Jon du Bois for allowing me to use his quadcopter model as a foundation for this research, as well as my supervisor Dr. Andrew Hillis for his continued guidance and support throughout this report. More personally, my family has supported me through my educational journey and enabled me to attend university, for which I am truly grateful. My wonderful girlfriend as well whose company I have cherished and treasured for over three years. Finally, our cats Atlas and Gaia have been a very necessary source of distraction and the likely culprits for any typos found hereinafter.

Contents

1 Introduction

The amount of unmanned aerial systems in the sky above us increases exponentially by the day as a result of their practicality, cost effectiveness, and ease of control. However, as the skies become increasingly congested, more safety is justly being demanded by regulative authorities. A key way to improving the safety and reliability of airborne UAS is to design a robust controller. Such a controller would be capable of recognizing the presence of a fault, isolating it, and assessing the impact of the fault, prior to taking mitigating action. An example of this would be a set of controllers capable of recognising the quadcopter's current flight dynamics, subsequently relating them to the state estimations of the various built-in controllers that it would be simulating at each specified time step. These controllers would be designed and optimised offline across a range of operating conditions, such as partial and full failure of one or more rotors, and would become the active controller if that operating condition were to materialise mid-flight. This could result in more accurate and precise autonomous control of the quadcopter especially in critical states.

The aim of this project is to create this robust quadcopter controller that is able to quickly and efficiently minimise the error between the UAV's actual position and a planned trajectory in spite of any faults that may be present within the system. Various controller types will be modelled and tested, with their performance analysed and compared. Model Predictive Control is hypothesised to provide favourable performance in this regard, especially in the case of actuator/rotor failure. This is because of the controllers intrinsic ability to manage dynamic constraints and a specified prediction and control horizon to provide optimal plant inputs. As the controller is developed in depth, areas that can be improved upon will be identified and novel measures implemented, with each successive change's influence on the response of the system documented and commented on.

The entirety of this research will be based on the mathematical model provided by Jonathan du Bois within the University of Bath academic circle [1]. A report that he has previously published established an intricate non-linear model as well as a linear version based on the hover state of the quadcopter. No effort is made hereinafter to take credit for the modelling work that he has done, this report merely seeks to research the best possible methods for *controlling* the system designed by du Bois in a variety of situations.

Several objectives of this project are listed below:

- **Create** MIMO (multiple input multiple output) **LQR** (Linear Quadratic Regulator) and **MPC** (Model Predictive Control) quadcopter control systems in three dimensions to interface with a predefined quadcopter model (plant).
- **Compare** both controllers in a variety of situations.
- **Implement** dynamic constraints such as physical limitations of the equipment, obstacles, and failure cases.
- **Improve** controller in novel ways to improve performance, robustness, and speed of the controller's response system.
- **Assess** overall importance of improvements.

1.1 Creation of initial LQR and MPC control systems

The first step to the creation of a controller is the mathematical model of the system that is to be controlled. This model contains all the relevant flight dynamics that transform a set of inputs (such as motor voltages) to a set of outputs (such as positions and velocities). Various matrices are therefore created to characterise the quadcopter's states, such as:

- Absolute linear position of the quadcopter in the inertial frame
- Attitude (angular position) of the quadcopter in the inertial frame
- Linear and angular position vectors
- Linear Velocities
- Angular Velocities
- Linear Accelerations
- Angular Accelerations
- Rotation matrix from the body frame to the inertial frame
- Transformation matrices for angular velocities between the inertial frame and the body frame
- Thrust in the body z-axis
- Torques in the corresponding body frame angles

In addition to these matrices, the Newton-Euler equations must be used to describe the quadcopter's dynamics and the Euler-Lagrange equations to derive the differential equation for the angular accelerations of the quadcopter. Additionally, the aerodynamic effects on the system can be included in varying levels of detail. On a base level, the drag force from air resistance can be modelled using drag coefficients with the quadcopter's velocities in the three dimensions.

The entirety of the control systems designed in this report, as well as the quadcopter model provided by [1], are assembled using Simulink, a MATLAB-based graphical programming environment for modelling, simulating and analysing multi-domain dynamic systems.

1.2 Comparison of controller types

In this stage various controllers will be designed and their performance tested in various simulations. A reference trajectory will be designed with enough complexity to be able to discern the performances of each controller. They will be compared based on several transient response specifications, such as delay time, rise time, peak time, settling time, and maximum overshoot. However, the performance of each controller type is more easily ascertained by visually evaluating its proximity to the reference signal that is superimposed upon the same graph. The most favourable controller is hypothesised to be MPC-based, and is therefore likely to be the controller that will be focused on going forward.

1.3 Implementation of constraints

A key feature of MPC is its ability to implement dynamic constraints, such as physical limitations of equipment, directly into the controller. Research is carried out into COTS electric brushless motors that are commonly used in quadcopter systems. However, as the provided mathematical model is based on a DJI F450, the A2208/A2212 800-1100 kV brushless motor is used, capable of 85-115 rad/s^{-1}V^{-1}. It will be assumed that the quadcopter will operate in urban environments where speed and safety are of the utmost importance, and that as such most of the on-board systems will be relevant to this type of mission.

In addition to this, several failure scenarios will be designed as opportunities to further test the controller. These range in severity from a sudden change in conditions to double rotor failure. The controller will have to dynamically adjust gains to reduce the sudden increase in its intrinsic cost function and be able to demonstrate its robustness as a whole.

1.4 Controller refinements

Much will be learned from the implementation of constraints as the controller is pushed to its physical and computational limits. During this process, various shortcomings and areas of improvement will be noted for the controller refinement phase. For the LQR controller, this is likely to involve the tuning of the `K` gain matrix by adjusting the weight matrices Q and R. For the MPC controller, improvements such as increases in the prediction and control horizon, and the simulation of multiple MPCs in parallel, will have to be balanced against corresponding increases in computational effort. This search for an optimal balance will also extend to the potential reduction of the time step used in the simulation of both controllers, which will also inevitably lead to an increase in computing time. Refinements will be brainstormed and implemented, with the impact of each refinement illustrated with a graph and its effect on computational cost documented as well. This will provide insight into whether the refinement is worth implementing into the final version of the controller.

2 Literature Review

2.1 The quadcopter model

A quadcopter is a common Linear Parameter Varying (LPV) system. LPVs are a special class of nonlinear systems which appear to be well suited for control of aerospace systems [2]. This is because there are many parameters (such as dynamic pressure, Mach number, angle of attack) that vary throughout flight. This results in an even larger amount of different operating conditions, for each of which exists a separate transfer function. Therefore, a tuned controller's gains are only applicable to a specific operating condition, e.g. hover. Gain scheduling is a common and easily implementable approach to solving this control problem. Various design points are considered (such as high Mach and low AoA, low Mach and high AoA), and if the system crosses a certain threshold in any relevant parameter, a separate set of gains is employed in the controller [3]. This controller theory is also applied to Adaptive MPC and Multiple MPC [4]. Other control methods that have been tested are cascaded PID [5], backstepping [6], nonlinear H_∞ [7], LQR [8], Model Reference Adaptive Control (MRAC) [9], and

Model Predictive Control (MPC) [10].

2.2 Fault detection, diagnosis, and control

Fault detection and diagnosis (FDD) and fault-tolerant control (FTC) play large parts in the design of robust controllers. Along with other safety-critical systems such as spacecraft, nuclear power plants, and chemical plants, the consequences of a minor component fault in an aircraft (manned or unmanned) can be catastrophic. It is therefore necessary to design control systems capable of tolerating potential faults automatically without compromising on performance. These are known as fault-tolerant control systems (FTCS) [11].

Many FDD approaches exist, namely model-based ones such as the Kalman Filter in its various forms [12], and data-based ones, such as neural networks and fuzzy logic [13]. Observer-based FDD techniques are also quite common [14]. Thau's observer, for example, has been employed to generate a set of residuals for detection and diagnosis of accelerometers and inclinometers faults [15] [16].

A successful FDD scheme must consecutively perform the following three tasks:

- **Fault Detection:** Recognize there is something wrong with the system
- **Fault Isolation:** Identify the location and type of fault
- **Fault Identification:** Determine the magnitude and severity of the fault

There is many criteria that can be used to compare various FDD methods, such as redundancy, robustness, nonlinearity, and stability, however one of the more important features to distinguish is its integration with reconfigurable control and FTC systems [11].

2.3 Controllers

Cascaded PID

Cascaded PID controllers are often used to provide basic control inputs to adjust roll, pitch and yaw. This design method is quite common in industry because of its functional ease and simplicity [17]. However, it does contain a few limitations, such as parameter uncertainty, external disturbances, and time delays [18]. In the case of actuator failures, sensor errors, and strong winds, cascaded PID is often not capable of mitigating resulting effects [19].

Model reference-based control

Model reference-based control is often used as a solution to FTC. This involves using a reference model that generates a desired trajectory and compares it to the actual trajectory to obtain various error models depending on whether passive FTC, active FTC or hybrid FTC is used. In passive FTC, faults are dealt with as though they were exogenous perturbations, as opposed

to active FTC, where the controller is scheduled by the fault estimation [20]. Hybrid FTC combines the characteristics of both whilst reducing their respective drawbacks. This is done by adjusting active FTC to more accurately estimate the uncertainty in the fault over time and thus reducing the effect that it has on the closed-loop response. As such, hybrid FTC is generally found to be the preferred option over PID [20].

LQR

Other methods used to handle rotor failures on quadrotors are LQR and LPV control, which are linear control methods. Feedback linearisation [21] [22] and incremental nonlinear dynamic inversion (INDI) [23] are also used, although they are nonlinear methods. However, each of these methods has its respective drawbacks.

LQR seems to be the preferred option to the optimal control problem. It is a feedback controller with an infinite or finite horizon, utilising either a discrete or continuous time variable [24]. Its aim is to minimise a cost function with weighting factors supplied by the designer of the controller, in the form of Q (state error) and R (actuator effort) matrices. This cost function will be defined as the sum of the deviations of the quadcopter's linear and angular positions and velocities, as well as the actuator effort required during the manoeuvre.

L1 adaptive control

L1 adaptive control has also been shown to be a noticeable improvement over cascaded PID [25]. In this control design method, parameters are systematically determined based on intuitively desired performance and robustness metrics set by the designer. It is a filtered version of Model Reference Adaptive Control (MRAC) that eliminates certain time delays by means of a low-pass filter.

$\mathbf{H}_\infty$

Another method of approaching the LPV control problem is to solve a set of linear matrix inequalities (LMIs) in order to achieve regional pole placement and H_∞ norm bounding constraints [20]. However, both MPC and conventional MRAC have been found to give better performance than H_∞ and simple LQR [14].

MPC

MPC control is hypothesised by the author to provide even better performance than LQR, albeit at the cost of increased computational time. This is because MPC is constantly optimising the plant inputs for the current horizon (see Fig. 1) at each time step. The optimisation is done by iteratively solving a model to reduce a specific cost function, often tied to the error between certain reference states and the corresponding system states. The prediction horizon keeps shifting forward resulting in a "receding horizon", a term in which MPC has also been referred as.

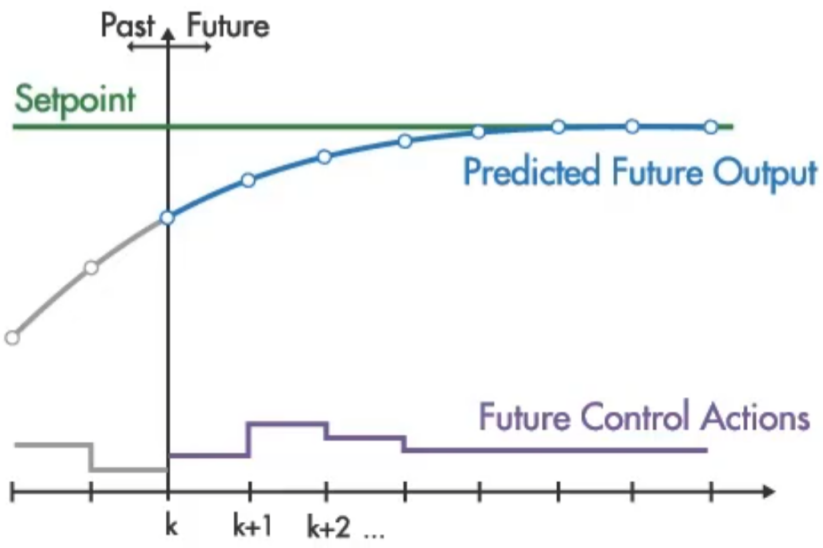


Figure 1: MPC Prediction and Control Horizon

Nonlinear MPC

Nonlinear MPC, or NLMPC, is a variant of MPC which is used when the plant model contains nonlinear dynamics. While the optimal control problems are convex in linear MPC, in NLMPC they are not necessarily convex, posing issues when it comes to stability theory and numerical solution [26] [27]. In these cases, the solvers within programming environments such as MATLAB are compatible with a Jacobian function as well as the base state derivatives to increase stability and computational efficiency. Additional tools are available which are discussed in more detail in section 4.5.

Although MPC is nearly universally implemented as a digital control, research is being done into achieving faster response times with specially designed analogue circuitry [28].

3 Mathematical Model

The model for the quadcopter is explained in this section. As stated previously, the quadcopter Simulink model is provided by Dr. Jon du Bois as a result of his work on quadcopter and cyclocopter flight dynamics [1]. Many of the following equations and derivations are taken directly from the theory section of his report as he constructed the model himself. No credit is taken by the author for any of the work depicted in sections 3.1 - 3.10, as it has already been done in [1]. The model has been approved for use by du Bois in the context of this FYP.

The general modelling framework is explained, comprising 16 quadcopter states, 3 coordinate frames, and the equations relating them. State derivatives, forces and torques are then established, with the physical properties of the DJI F450 being outlined as well. Finally, the model is linearised around the hover point and the controllers to be tested are detailed.

3.1 Modelling Framework [1]

The absolute linear position of the quadcopter is defined in the inertial earth frame axes x, y, and z (see Fig. 2). The attitude (angular position) is defined in the inertial frame with three Euler angles ψ, θ, and ϕ. The vehicle's mass properties are given in Table 1.

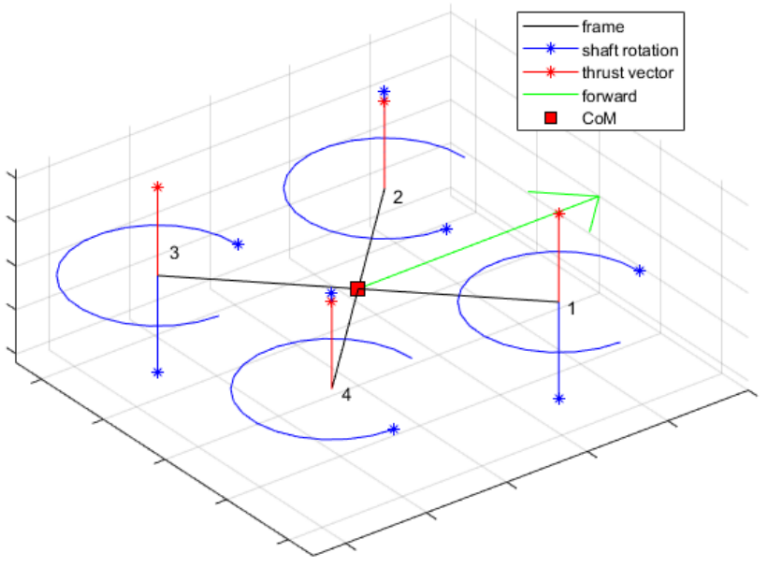


Figure 2: Quadcopter rotor layout, with stars representing arrowheads on thrust and rotation vectors [1]

Table 1: Quadcopter model physical properties

Parameter	Symbol	Value	Units
Body Mass	m_b	0.760	kg
Individual Rotor Mass	m_r	0.0599	kg
Distance from the centre of mass to each rotor	L_r	0.225	m
Rotor moment of inertia about axis normal to shaft	$I_{\perp}^{(r)}$	$1.11 \cdot 10^{-4}$	kgm^2
Rotor moment of inertia about shaft axis	$I_{zz}^{(r)}$	$2.23 \cdot 10^{-4}$	kgm^2
Body moment of inertia about x axis (longitudinal axis)	$I_{xx}^{(b)}$	$9.42 \cdot 10^{-4}$	kgm^2
Body moment of inertia about y axis (lateral axis)	$I_{yy}^{(b)}$	$9.42 \cdot 10^{-4}$	kgm^2
Body moment of inertia about z axis (vertical axis)	$I_{zz}^{(b)}$	$1.87 \cdot 10^{-4}$	kgm^2

The state vector for the aircraft model in Eq. 1 consists of 16 states, listed below the following equation.

$$\begin{pmatrix} u & v & w & p & q & r & x & y & z & \phi & \theta & \psi & w_{R_1} & w_{R_2} & w_{R_3} & w_{R_4} \end{pmatrix}^T \tag{1}$$

- u, v, w: Linear velocities in the x, y and z axes of the earth frame.
- p, q, r: Angular velocities along ϕ, θ, and ψ in the body frame.
- x, y, z: Positions in the earth frame.
- ϕ, θ, ψ: Euler angles (positions) in the body frame.

- w_{R_i}: Angular velocity of the ith rotor.

In addition to the Earth frame (E) and body frame (B), an additional intermediate frame (I) is to be considered. It is an inertial reference frame aligned instantaneously with the body frame. The states used in the intermediate frame shown in Eq. 2 are listed below the equation.

$$\begin{pmatrix} I_\xi & I_{\dot{\xi}} & I_\eta & I_{\dot{\eta}} & \psi_r & \dot{\psi}_r \end{pmatrix}^T \tag{2}$$

- I_ξ: Position in the intermediate frame.
- $I_{\dot{\xi}}$: Velocity in the intermediate frame.
- I_η: Euler angle vector in the intermediate frame.
- $I_{\dot{\eta}}$: Euler angle derivatives in the intermediate frame.
- ψ_r: Vector of rotor angles.
- $\dot{\psi}_r$: Vector of rotor velocities.

It is useful to illustrate the relationships between the three coordinate frames used to compute and record the quadcopter's dynamics. Eq. 3 demonstrates the relationship between the Earth frame and body frame, Eq. 6 between the intermediate and body frames, and Eq. 8 between the intermediate and earth frames.

$$E_{\dot{\xi}} = E_{R_B} B_v \quad E_{\ddot{\xi}} = E_{\dot{R}_B} B_v + E_{R_B} B_{\dot{v}} \quad E_{\dot{\eta}} = E_{H_B} B_\omega \quad E_{\ddot{\eta}} = E_{\dot{H}_B} B_\omega + E_{H_B} B_{\dot{\omega}} \tag{3}$$

where

$$H = \begin{pmatrix} 1 & 0 & -s_\theta \\ 0 & c_\phi & c_\theta s_\phi \\ 0 & -s_\phi & c_\phi c_\theta \end{pmatrix}^{-1} = \frac{1}{c_\theta} \begin{pmatrix} c_\theta & s_\phi s_\theta & c_\phi s_\theta \\ 0 & c_\theta c_\phi & -c_\theta s_\phi \\ 0 & s_\phi & c_\phi \end{pmatrix} \tag{4}$$

The direction cosine matrix is shown in Eq. 5, where the shorthand $c_\bullet = \cos(\bullet)$ and $s_\bullet = \sin(\bullet)$ is used:

$$E_{R_B} = \begin{pmatrix} c_\psi c_\theta & c_\psi s_\phi s_\theta - c_\phi s_\psi & s_\phi s_\psi + c_\phi c_\psi s_\theta \\ c_\theta s_\psi & c_\phi c_\psi + s_\phi s_\psi s_\theta & c_\phi s_\psi s_\theta - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\phi c_\theta \end{pmatrix} \tag{5}$$

The relationship between the intermediate and body frames is given by:

$$I_{\dot{\xi}} = I_{R_B} B_v \quad I_{\ddot{\xi}} = I_{\dot{R}_B} B_v + I_{R_B} B_{\dot{v}} \quad I_{\dot{\eta}} = I_{H_B} B_\omega \quad I_{\ddot{\eta}} = I_{\dot{H}_B} B_\omega + I_{H_B} B_{\dot{\omega}} \tag{6}$$

where $I_\eta = 0$ so

$$I_{\dot{\xi}} = B_v \quad I_{\ddot{\xi}} = \begin{pmatrix} qw - rv \\ ru - pw \\ pv - qu \end{pmatrix} + B_{\dot{v}} \quad I_{\dot{\eta}} = B_\omega \quad I_{\ddot{\eta}} = \begin{pmatrix} qr \\ -pr \\ pq \end{pmatrix} + B_{\dot{\omega}} \tag{7}$$

Finally, the intermediate and Earth frames are related by:

$$E_{\dot{\xi}} = E_{R_B} I_{\dot{\xi}} \quad E_{\dot{\eta}} = E_{H_B} I_{\dot{\eta}} \tag{8}$$

The equations of motions for the quadcopter can then be derived using the intermediate frame (I), with the system represented using the state vector shown in Eq. 9.

$$\mathbf{z} = \begin{pmatrix} I_\xi \\ I_\eta \\ \psi_r \\ I_{\dot{\xi}} \\ I_{\dot{\eta}} \\ \dot{\psi}_r \end{pmatrix} \tag{9}$$

3.2 Conventional Aircraft States [1]

Conventional aircraft states are derived from the intermediate states in Eq. 10, which will be refined in the following subsection.

$$\begin{pmatrix} B_{\dot{v}} \\ B_{\dot{w}} \\ E_{\dot{\xi}} \\ E_{\dot{\eta}} \\ \dot{\omega}_r \end{pmatrix} = \begin{pmatrix} I_{\ddot{\xi}} - \begin{pmatrix} qw - rv \\ ru - pw \\ pv - qu \end{pmatrix} \\ I_{\ddot{\eta}} - \begin{pmatrix} qr \\ -pr \\ pq \end{pmatrix} \\ E_{R_B} B_v \\ E_{H_B} B_w \\ \ddot{\psi}_r \end{pmatrix} \tag{10}$$

The intermediate states in the expressions for $I_{\ddot{\xi}}$, $I_{\dot{\eta}}$ and $\ddot{\psi}_r$ shown in Eq. 7 are eliminated using the substition in Eq. 11.

$$\begin{pmatrix} I_\xi \\ I_{\dot{\xi}} \\ I_\eta \\ I_{\dot{\eta}} \\ \psi_r \\ \dot{\psi}_r \end{pmatrix} = \begin{pmatrix} 0 \\ B_v \\ 0 \\ B_\omega \\ 0 \\ \omega_r \end{pmatrix} \tag{11}$$

Note that concerning the rotor position states ψ_r and rotor velocity states $\dot{\psi}_r$, only the rotor velocity states have any influence on the state derivatives, and is therefore included in the final system state under the form $\omega_r = \dot{\psi}_r$, whereas the rotor position is omitted.

3.3 Forces and Torques [1]

The forces and torques acting on the quadcopter can be separated into three components: the body force and moments, the motor torques and rotor aerodynamics, and gravity. These are shown in Eq. 12.

$$I_F = \begin{pmatrix} F_x \\ F_y \\ F_z \\ F_\phi \\ F_\theta \\ F_\psi \\ F_R \end{pmatrix} = \begin{pmatrix} X \\ Y \\ Z \\ L \\ M \\ N \\ \mathbf{0} \end{pmatrix} + G \begin{pmatrix} \mathbf{M} \\ \mathbf{T} \\ \mathbf{H} \\ \mathbf{Y} \\ \mathbf{Q} \\ \mathbf{M_x} \\ \mathbf{M_y} \end{pmatrix} + \begin{pmatrix} (E_{R_B})^{-}1 \begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix} \\ 0 \\ 0 \\ 0 \\ \mathbf{0} \end{pmatrix} \tag{12}$$

where

- X, Y, and Z: Body aerodynamic forces.
- L, M, and N: Body aerodynamic moments.
- $\mathbf{M}$: Vector of n motor torques acting on the rotor shafts.
- $\mathbf{T}$: Vector of n motor thrusts produced by each rotor (normal to the rotor plane).
- $\mathbf{H}$: Vector of n drag forces on each rotor (in the $-x$ direction).
- $\mathbf{Y}$: Vector of n lateral forces on each rotor (in the y direction).
- $\mathbf{Q}$: Vector of n aerodynamic torques about the rotor shaft opposing the direction of rotation.
- $\mathbf{M_x}$ and $\mathbf{M_y}$: Aerodynamic moments acting on the rotors about the x- and y- axes in the rotor frame
- m: Mass of the vehicle.
- G: 10 x 28 matrix relating rotor forces and model inputs (defined in Eq.19).

3.4 Intermediate Frame [1]

The intermediate frame states described in Eq. 9 are used to obtain the equations of motion:

$$\begin{pmatrix} M_1 & 0 \\ 0 & M_2 \end{pmatrix} \ddot{\mathbf{x}} + \begin{pmatrix} \dot{\mathbf{x}}^{\mathbf{T}} C_1 \dot{\mathbf{x}} \\ \vdots \\ \dot{\mathbf{x}}^{\mathbf{T}} C_{10} \dot{\mathbf{x}} \end{pmatrix} = I_F \tag{13}$$

where I_F includes the gravity force, aerodynamic body and rotor forces/moments, and the motor torques stated in Eq. 12. To better understand the previous equation, Eq. 9 can be split as follows:

$$\mathbf{z} = \begin{pmatrix} \mathbf{x} \\ \dot{\mathbf{x}} \end{pmatrix} = \begin{pmatrix} \begin{pmatrix} I_\xi \\ I_\eta \\ \psi_r \end{pmatrix} \\ \begin{pmatrix} I_{\dot{\xi}} \\ I_{\dot{\eta}} \\ \dot{\psi}_r \end{pmatrix} \end{pmatrix} \tag{14}$$

The mass matrix is comprised of a diagonal matrix M_1 for the five uncoupled body degrees of freedom and a symmetric matrix M_2 representing the rotor degrees of freedom and the coupled degree of freedom for the body. These are shown in Eqs. 15a and 15b.

$$\begin{pmatrix} m_b + 4m_r & 0 & 0 & 0 & 0 \\ 0 & m_b + 4m_r & 0 & 0 & 0 \\ 0 & 0 & m_b + 4m_r & 0 & 0 \\ 0 & 0 & 0 & 2m_r L_r^2 + 4I_\perp^{(r)} + I_{xx}^{(b)} & 0 \\ 0 & 0 & 0 & 0 & 2m_r L_r^2 + 4I_\perp^{(r)} + I_{yy}^{(b)} \end{pmatrix} \tag{15a}$$

$$\begin{pmatrix} 4m_r L_r^2 + I_{zz}^{(b)} + I_{zz}^{(r)} & I_{zz}^{(r)} & -I_{zz}^{(r)} & I_{zz}^{(r)} & -I_{zz}^{(r)} \\ I_{zz}^{(r)} & I_{zz}^{(r)} & 0 & 0 & 0 \\ -I_{zz}^{(r)} & 0 & I_{zz}^{(r)} & 0 & 0 \\ I_{zz}^{(r)} & 0 & 0 & I_{zz}^{(r)} & 0 \\ -I_{zz}^{(r)} & 0 & 0 & 0 & I_{zz}^{(r)} \end{pmatrix} \tag{15b}$$

The Coriolis forces for the system must also be considered. They are defined in Eqs. 16a - 16e.

$$C_1 = C_2 = C_3 = 0 \tag{16a}$$

$$C_4 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{I_{zz}^{(b)}}{2} - \frac{I_{xx}^{(b)}}{2} - \frac{I_{yy}^{(b)}}{2} - 4I_\perp^{(r)} + 2I_{zz}^{(r)} & \frac{I_{zz}^{(r)}}{2} & -\frac{I_{zz}^{(r)}}{2} & \frac{I_{zz}^{(r)}}{2} & -\frac{I_{zz}^{(r)}}{2} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{16b}$$

$$C_5 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 4I_{\perp}^{(r)} + \frac{I_{xx}^{(b)}}{2} + \frac{I_{yy}^{(b)}}{2} - \frac{I_{zz}^{(b)}}{2} - 2I_{zz}^{(r)} & -\frac{I_{zz}^{(r)}}{2} & \frac{I_{zz}^{(r)}}{2} & -\frac{I_{zz}^{(r)}}{2} & \frac{I_{zz}^{(r)}}{2} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{16c}$$

$$C_6 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -2m_r L_r^2 - \frac{I_{xx}^{(b)}}{2} + \frac{I_{yy}^{(b)}}{2} - \frac{I_{zz}^{(b)}}{2} - 2I_{zz}^{(r)} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{16d}$$

$$C_7 = -C_8 = C_9 = -C_{10} = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\frac{I_{zz}^{(r)}}{2} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\frac{I_{zz}^{(r)}}{2} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{16e}$$

The roll and pitch rates of the quadcopter therefore couple with the rotor speeds to produce precession torques. This results in the quadcopter describing a cone in space when torque is applied. However these effects are negligible and will not be included in the final quadcopter model.

3.5 Aircraft States [1]

The quadcopter's 16 state derivatives are stated in Eq. 17. These are derived from Eqs. 10 and 11 as well as the rotor state derivates in Eqs 13 to 16.

$$\dot{u} = rv - qw + \frac{F_x}{m_b + 4m_r} \tag{17a}$$

$$\dot{v} = pw - ru + \frac{F_y}{m_b + 4m_r} \tag{17b}$$

$$\dot{w} = qu - pv + \frac{F_z}{m_b + 4m_r} \tag{17c}$$

$$\dot{p} = \frac{T_\phi + qI_{zz}^{(r)}(-\omega_{r1} + \omega_{r2} - \omega_{r3} + \omega_{r4} + qr(-2m_r L_r^2 + I_\perp^{(b)} + 4I_\perp^{(r)} - I_{zz}^{(b)} - 4I_{zz}^{(r)}}{2m_r L_r^2 + I_p^{(b)} erp + 4I_\perp^{(r)}} \tag{17d}$$

$$\dot{q} = \frac{T_\theta + pI_{zz}^{(r)}(\omega_{r1} - \omega_{r2} + \omega_{r3} - \omega_{r4} + pr(2m_r L_r^2 - I_\perp^{(b)} - 4I_\perp^{(r)} + I_{zz}^{(b)} + 4I_{zz}^{(r)}}{2m_r L_r^2 + I_p^{(b)} erp + 4I_\perp^{(r)}} \tag{17e}$$

$$\dot{r} = \frac{T_\psi + T_{r2} - T_{r1} - T_{r3} + T_{r4}}{4m_r L_r^2 + I_{zz}^{(b)}} \tag{17f}$$

$$\dot{x} = c_\psi c_\theta u - c_\phi s_\psi v + s_\phi s_\psi w + c_\phi c_\psi s_\theta w + c_\psi s_\phi s_\theta v \tag{17g}$$

$$\dot{y} = c_\phi c_\psi v + c_\theta s_\psi u - c_\psi s_\phi w + c_\phi s_\psi s_\theta w + s_\phi s_\psi s_\theta v \tag{17h}$$

$$\dot{z} = c_\phi c_\theta w = s_\theta u + c_\theta s_\phi v \tag{17i}$$

$$\dot{\phi} = \frac{c_\theta p + c_\phi r s_\theta + q s_\phi s_\theta}{c_\theta} \tag{17j}$$

$$\dot{\theta} = c_\phi q - r s_\phi \tag{17k}$$

$$\dot{\psi} = \frac{c_\phi r + q s_\phi}{c_\theta} \tag{17l}$$

$$\dot{\omega}_{r1} = \frac{T_{r1}}{I_{zz}^{(r)}} + \frac{-T_\psi + T_{r1} - T_{r2} + T_{r3} - T_{r4}}{4m_r L_r^2 + I_{zz}^{(b)}} \tag{17m}$$

$$\dot{\omega}_{r2} = \frac{T_{r2}}{I_{zz}^{(r)}} + \frac{T_\psi - T_{r1} + T_{r2} - T_{r3} + T_{r4}}{4m_r L_r^2 + I_{zz}^{(b)}} \tag{17n}$$

$$\dot{\omega}_{r3} = \frac{T_{r3}}{I_{zz}^{(r)}} + \frac{-T_\psi + T_{r1} - T_{r2} + T_{r3} - T_{r4}}{4m_r L_r^2 + I_{zz}^{(b)}} \tag{17o}$$

$$\dot{\omega}_{r4} = \frac{T_{r4}}{I_{zz}^{(r)}} + \frac{T_\psi - T_{r1} + T_{r2} - T_{r3} + T_{r4}}{4m_r L_r^2 + I_{zz}^{(b)}} \tag{17p}$$

3.6 Force inputs [1]

The forces acting on the aircraft stated in Eq. 12 are then:

$$\begin{pmatrix} F_x \\ F_y \\ F_z \\ F_\phi \\ F_\theta \\ F_\psi \\ T_{r1} \\ T_{r2} \\ T_{r3} \\ T_{r4} \end{pmatrix} = \begin{pmatrix} X \\ Y \\ Z \\ L \\ M \\ N \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + G \begin{pmatrix} \mathbf{M} \\ \mathbf{T} \\ \mathbf{H} \\ \mathbf{Y} \\ \mathbf{Q} \\ \mathbf{M_x} \\ \mathbf{M_y} \end{pmatrix} + \begin{pmatrix} (E_{R_B})^-1 \begin{pmatrix} 0 \\ 0 \\ mg \end{pmatrix} \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \tag{18}$$

With the matrix G is defined in Eq. 19:

$$G = \begin{pmatrix}
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & -1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & -\frac{\sqrt{2}L_r}{2} & \frac{\sqrt{2}L_r}{2} & \frac{\sqrt{2}L_r}{2} & -\frac{\sqrt{2}L_r}{2} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \frac{\sqrt{2}L_r}{2} & \frac{\sqrt{2}L_r}{2} & -\frac{\sqrt{2}L_r}{2} & -\frac{\sqrt{2}L_r}{2} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{\sqrt{2}L_r}{2} & -\frac{\sqrt{2}L_r}{2} & -\frac{\sqrt{2}L_r}{2} & \frac{\sqrt{2}L_r}{2} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
\end{pmatrix}$$

$$\begin{pmatrix}
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\
\frac{\sqrt{2}L_r}{2} & \frac{\sqrt{2}L_r}{2} & -\frac{\sqrt{2}L_r}{2} & -\frac{\sqrt{2}L_r}{2} & -1 & -1 & -1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
\end{pmatrix} \tag{19}$$

3.7 Rotor Forces [1]

The rotor force and moment coefficients in the XZ plane can be approximated using blade element theory as a function of the advance ratio components μ_x and μ_z [29].

$$C_T = \frac{\sigma C_L \alpha}{2}\left[\theta_0\left(\frac{2}{3}+\mu_x^2\right) - (\mu_z + \lambda_i)\right] \tag{20a}$$

$$C_H = \sigma\left[C_{L\alpha}(\mu_z+\lambda_i)\frac{\mu_x}{2}\theta_0 + \frac{1}{2}\mu_x C_D\right] \tag{20b}$$

$$C_Q = \frac{\sigma C_L \alpha}{2}(\mu_z+\lambda_i)\left[\frac{2}{3}\theta_0 - (\mu_z+\lambda_i)\right] + \frac{\sigma}{4}C_D(1+\mu_x^2) \tag{20c}$$

$$C_M = -\frac{\sigma C_L \alpha}{2}\left[\frac{2}{3}\theta_0\mu_x - \frac{1}{2}\mu_x(\mu_z+\lambda_i)\right] \tag{20d}$$

where C_T is the thrust in the $-z$ direction, C_H is the drag in the $-x$ direction, C_Q is the torque opposing rotation, and C_M is the rolling moment about the x axis. $C_{L\alpha}$ is the aerofoil lift curve, C_D is the aerofoil drag coefficient, θ_0 is the blade pitch, $\sigma = 2cN/\pi D$ is the solidity ratio, and λ_i is the induced velocity.

The four aerofoil coefficients are fitted to data from XFOIL [30], for a symmetric NACA0012 aerofoil which is similar enough to the quadcopter rotor to give representative results. The

constant aerofoil incidence θ_0 used in the blade element theory is described in Eq. 21 as a function of the pitch P and diameter D at 75% span incidence. The solidity is given by Eq. 22 where c is the blade chord and N is the number of blades.

$$\theta_0 = \text{atan}\left(\frac{P}{D\pi \cdot 0.75}\right) \tag{21}$$

$$\sigma = \frac{2cN}{\pi D} \tag{22}$$

A uniform induced velocity λ_i was approximated using an extended momentum theory based on a cubic curve fit for the VR/turbulent wake region as specified in [31] and [32]. This gives an induced velocity related to the advance ratio components μ_x and μ_z and the equivalent hover induced velocity λ_h by the surface shown in Fig. 3. The coefficient surfaces in Fig. 4 are then found by iteratively solving Eqs. 20 for a range of μ_x and μ_z. The assumptions used deteriorate for large advance ratios therefore the study is limited to $\mu_{x,z} = \pm 0.3$.

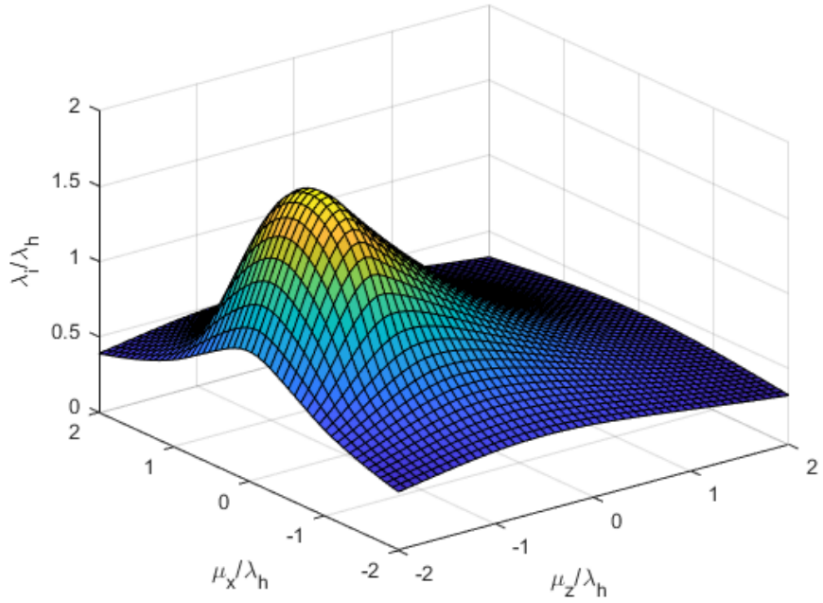
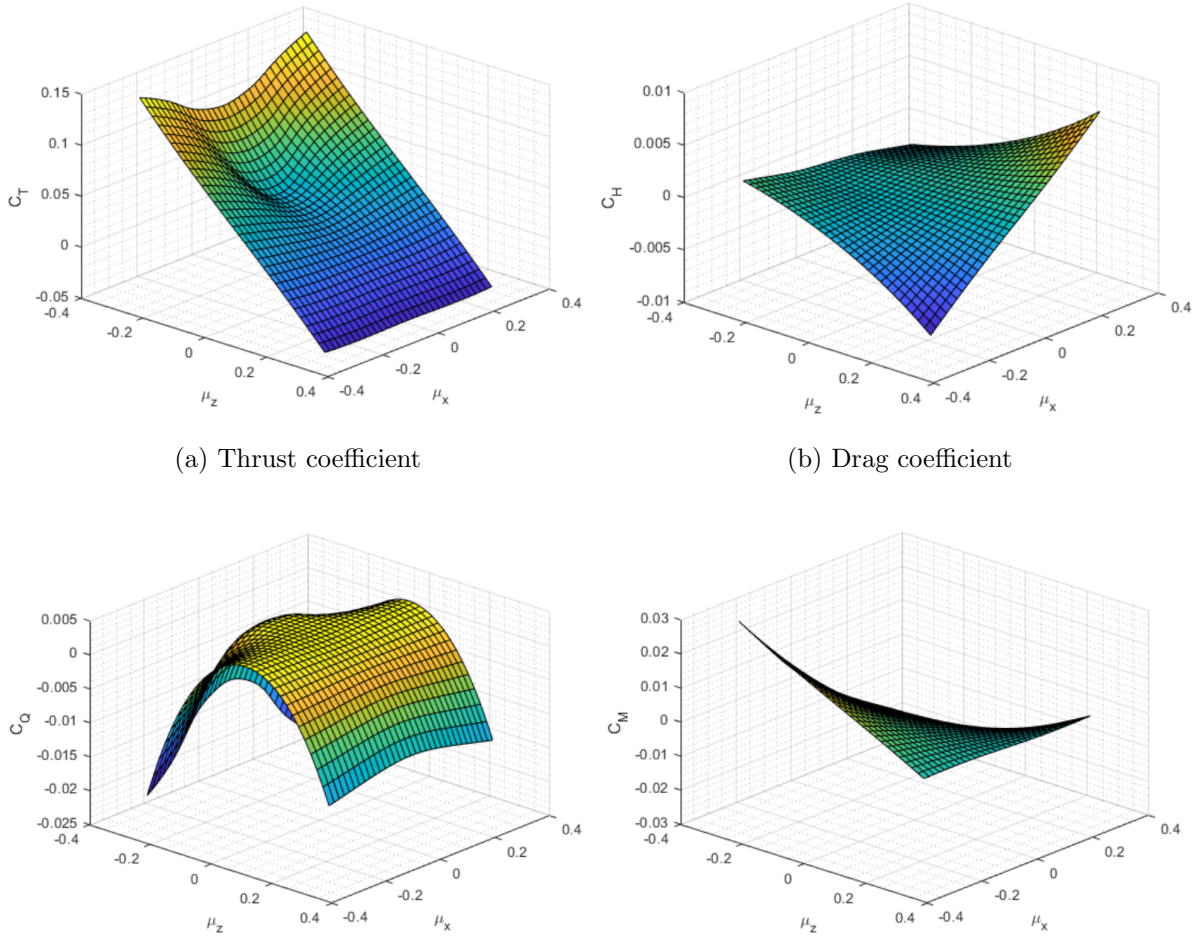


Figure 3: Extended momentum theory for induced velocity λ_i as a function of advance ratio components μ_x and μ_z, as well as hover induced velocity λ_h [1]

The relative wind with respect to the rotor in three dimensions can be transformed to a coordinate frame where it lies in an XZ plane (z aligned with the rotor z-axis; normally the shaft for a quadcopter rotor), and the 2 forces and 2 moments determined using a lookup table for the resultant μ_x and μ_z. These forces and moments are then transformed back to the 3 forces and 3 moments in the rotor frame where they can be used in Eq. 12.

If the vehicle velocity in the body frame is B_v and the wind velocity in the earth frame is E_w, then the relative freestream velocity in the body frame is:

$$B_{w_\infty} = \begin{pmatrix} B_{w_\infty}^{(x)} \\ B_{w_\infty}^{(y)} \\ B_{w_\infty}^{(z)} \end{pmatrix} = -B_v + E_{R_B}^{-1} E_w \tag{23}$$



(a) Thrust coefficient

(b) Drag coefficient

(c) Torque coefficient

(d) Rolling moment coefficient

Figure 4: Quadcopter rotor force and moment coefficients for a range of advance ratio components μ_x and μ_z [1]

The freestream velocity is transformed to the rotor frame using

$$R_{w_\infty} = B_{R_R}^{-1} B_{w_\infty} \tag{24}$$

A new frame, the *wind frame*, is now defined, denoted with W sub- and super-scripts. The z-axis is aligned with the rotor shaft and ${}^{W}w_\infty$ lies in the XZ-plane such that the rotation from the rotor frame to the wind frame is given by

$${}^{R}R_W = \frac{1}{\sqrt{{}^{R}w_\infty^{(x)^2} + {}^{R}w_\infty^{(y)^2}}} \begin{pmatrix} {}^{R}w_\infty^{(x)} & -{}^{R}w_\infty^{(y)} & 0 \\ {}^{R}w_\infty^{(y)} & {}^{R}w_\infty^{(x)} & 0 \\ 0 & 0 & \sqrt{{}^{R}w_\infty^{(x)^2} + {}^{R}w_\infty^{(y)^2}} \end{pmatrix} \tag{25}$$

The freestream velocity can then be computed in the wind frame with

$${}^{W}w_\infty = {}^{R}R_W^{-1} \cdot {}^{R}w_\infty \tag{26}$$

With the advance ratio components in this frame being

$$ {}^{W}\mu_x = \frac{-{}^{W}w_\infty^{(x)}}{\Omega R_{rotor}} \qquad {}^{W}\mu_z = \frac{-{}^{W}w_\infty^{(z)}}{\Omega R_{rotor}} \tag{27} $$

Using these in Eqs. 20 yields ${}^{W}C_T$, ${}^{W}C_H$,${}^{W}C_Q$, and ${}^{W}C_M$ which are transformed to the rotor frame using

$$ \begin{pmatrix} -{}^{R}C_H \\ {}^{R}C_Y \\ -{}^{R}C_T \end{pmatrix} = {}^{R}R_W \begin{pmatrix} -{}^{W}C_H \\ 0 \\ -{}^{W}C_T \end{pmatrix} \tag{28a} $$

$$ \begin{pmatrix} {}^{R}C_{M_x} \\ {}^{R}C_{M_y} \\ {}^{R}C_Q \end{pmatrix} = {}^{R}R_W \begin{pmatrix} {}^{W}C_M \\ 0 \\ {}^{W}C_Q \end{pmatrix} \tag{28b} $$

These coefficients are multiplied by $\frac{1}{2}\rho(\Omega R)^2 A$ (for the forces) $\frac{1}{2}\rho(\Omega R)^2 AR$ (for the torques), and adjusted for spin direction (M negated for positive spin) to give the rotor force and torque inputs used in Eq. 12.

3.8 Body Forces [1]

The effects of aerodynamic drag are modelled using a simple drag law. The freestream flow experienced by the aircraft is given in Eq. 23 and the freestream unit vector is defined as

$$ {}^{B}\hat{w}_\infty = {}^{B}w_\infty / ||{}^{B}w_\infty|| \tag{29} $$

The force vector acting on the body in the body frame is then

$$ \begin{aligned} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} &= \frac{1}{2}\rho||{}^{B}w_\infty||^2 A C_d {}^{B}\hat{w}_\infty \\ &= \frac{1}{2}\rho||{}^{B}w_\infty|| A C_d {}^{B}w_\infty \end{aligned} $$

where the aggregate properties assumed for A and C_d are 0.0134 m^2 and 0.75 for both aircraft.

3.9 Motors and Actuation [1]

Stevens *et al.* [33] give an expression for the motor torque as

$$Q_m = \frac{1}{K_v R_{wind}}(v - \frac{\omega_r}{K_v}) \tag{30}$$

where K_v is the motor speed constant, R_{wind} is the winding resistance, v is the voltage. The same speed constant also relates the current i and the torque:

$$Q_m = \frac{i}{K_v} \tag{31}$$

Rearranging gives

$$\frac{v}{\omega_r} = R_{wind}\frac{i}{\omega_r} - \frac{1}{K_v} \tag{32}$$

Plotting v/ω_r against i/ω_r and fitting a linear trendline gives estimates for the winding resistance and speed constant, as seen in Fig. 5 [1].

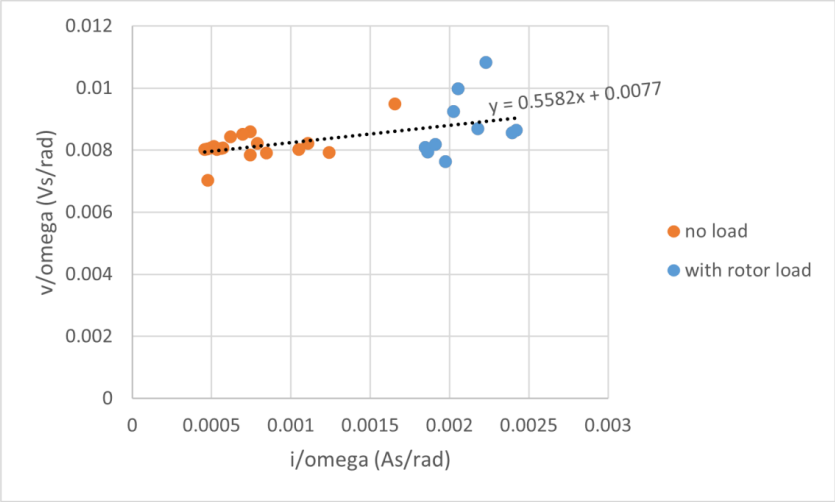


Figure 5: Experimental identification of motor constants, giving $K_v = 130$ rad/Vs (= 1240 RPM/V) and $R_{wind} = 0.56\Omega$ [1]

3.10 Model Linearisation [1]

Initially, the force and moment coefficient maps shown in Fig. 4 are linearised about the trim (hover) condition, taken as 0.25kg per rotor (total weight 1kg). This corresponds to an operating point of 4386 RPM (459.3 rad/s). The force and moment coefficients are described as

$$^W C_\bullet = {}^W C_{\bullet,trim} + \frac{\partial {}^W C_\bullet}{\partial \mu_x}(\mu_x - \mu_{x,trim}) + \frac{\partial {}^W C_\bullet}{\partial \mu_z}(\mu_z - \mu_{z,trim}) \tag{33}$$

where $\bullet$ is either T, H, M, or Q. Taking partial derivatives as constants at the trim condition results in a linear relationship between the advance ratio components and the force/moment

coefficients. Combining this linear relationship from Eq. 33 with the equations developed in section 3 allows for the creation of analytic nonlinear state space models for the quadcopter, illustrated in Table 2. Note that cyclic pitch actuation dynamics are neglected in this report. Taking Jacobians of the resulting nonlinear state derivatives with respect to the states and the inputs, and substituing the trim conditions, yields the linear state space matrices A and B given in Appendix A.

Table 2: Nonlinear state space model definition

States	Inputs	Parameters	Disturbances
u	V_1 (motor 1 voltage input)	A (rotor)	${}^{E}w_{\infty}^{(x)}$
v	V_2 (motor 2 voltage input)	A_{body}	${}^{E}w_{\infty}^{(y)}$
w	V_3 (motor 3 voltage input)	C_{Dbody}	${}^{E}w_{\infty}^{(z)}$
p	V_4 (motor 4 voltage input)	$I_{\perp}^{(b)}$	
q		$I_{zz}^{(b)}$	
r		$I_{\perp}^{(r)}$	
x		$I_{zz}^{(r)}$	
y		K_v	
z		m_b	
ϕ		m_r	
θ		R (rotor)	
ψ		L_r	
ω_{r1}		R_{wind}	
ω_{r2}		g (gravity cst.)	
ω_{r3}		ρ (air density)	
ω_{r4}			

It should also be noted that where ${}^{W}w_{\infty}^{(x)} = {}^{W}w_{\infty}^{(y)} = 0$, the quadcopter rotor frame rotation matrix stated in Eq. 25 is numerically problematic, as the wind frame is undefined for this specific condition. This inconsistency manifests itself in the form of a singularity in the Jacobians used to derive the linear models rather than the nonlinear state derivates themselves from which the Jacobians are taken. The coefficients are therefore written directly in the rotor frame as

$$ {}^{R}C_T = {}^{W}C_{T,trim} + \frac{\partial {}^{W}C_T}{\partial \mu_z}(\mu_z - \mu_{z,trim}) \tag{34a} $$

$$ {}^{R}C_H = \frac{\partial {}^{W}C_H}{\partial \mu_x}\mu_x \tag{34b} $$

$$ {}^{R}C_Y = -\frac{\partial {}^{W}C_H}{\partial \mu_x}\mu_y \tag{34c} $$

$$ {}^{R}C_Q = {}^{W}C_{Q,trim} + \frac{\partial {}^{W}C_Q}{\partial \mu_z}(\mu_z - \mu_{z,trim}) \tag{34d} $$

$$ {}^{R}C_{M_x} = \frac{\partial {}^{W}C_M}{\partial \mu_x}\mu_x \tag{34e} $$

$$ {}^{R}C_{M_y} = \frac{\partial {}^{W}C_M}{\partial \mu_x}\mu_y \tag{34f} $$

where μ_x, μ_y and μ_z are in this case defined in the rotor frame using right-handed conventions, and the partial derivatives are the same ones used in Eq. 33. From here the analytic nonlinear

state derivative equations can be constructed as before and the Jacobians computed to yield the linear state space matrices A and B stated in App. A.

4 Results & Analysis

As stated in the theory section, the linear model is used prior to the non-linear model to adequately assess the performance of the LQR and MPC controllers. The linear properties of the input-to-output relationships help create an initial understanding of the system's response in different situations. Once these foundations are established the non-linear model will be analysed and its salient features outlined.

4.1 LQR

A basic closed-loop LQR control system is initially evaluated. The system diagram created in the MATLAB Simulink environment is shown in Fig. 6.

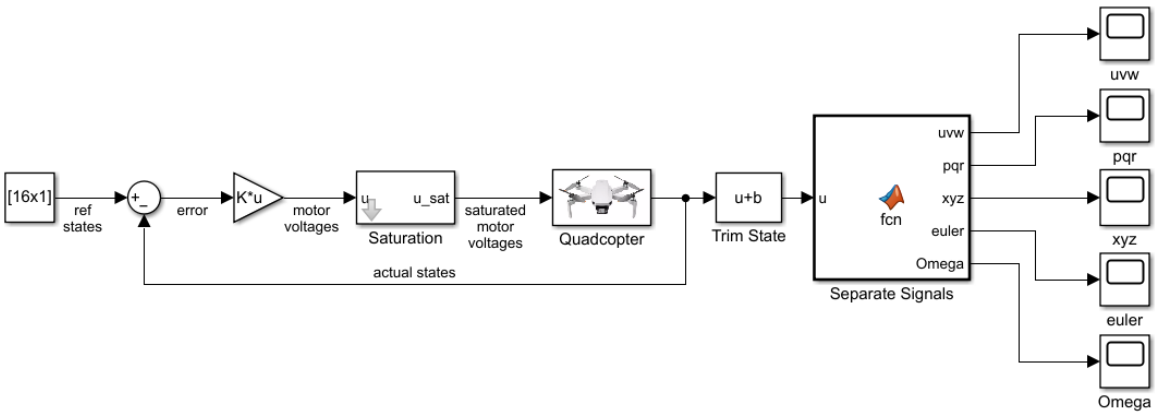


Figure 6: Diagram of Linear Quadcopter Model with LQR controller

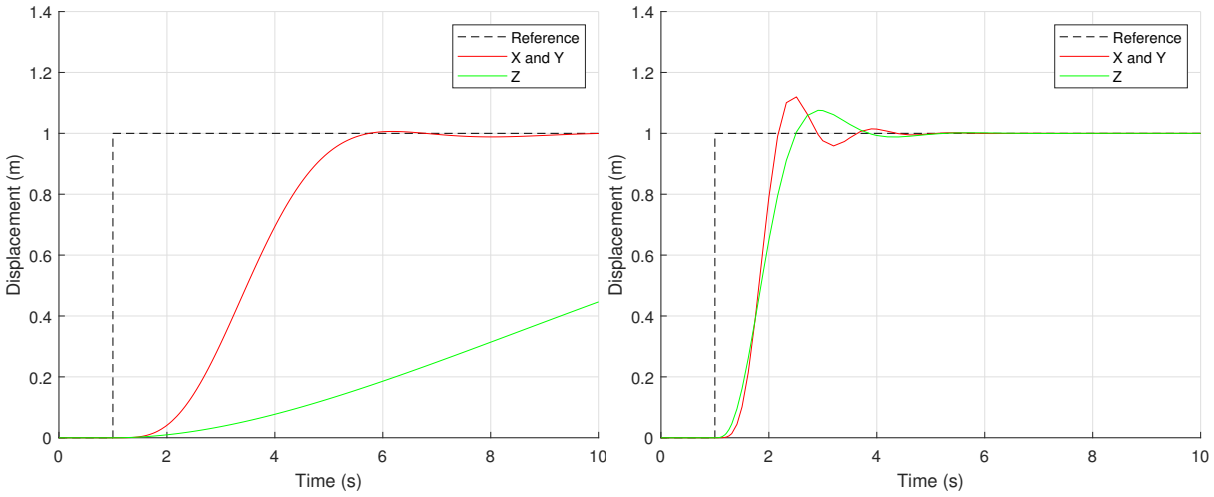
The 16 reference states are all set to 0, save for the position states 7:9 (x, y, and z) which are set to 1. The step block that actives the reference signal mid-simulation is omitted from the diagram for simplicity. The quadcopter block is a linear steady-state system, described as the steady state equation (35), with state matrices A and B (see App. A), the C matrix set to a diagonal unitary 16x16 matrix, and the D matrix set to 0.

$$\dot{x} = Ax + Bu \tag{35a}$$
$$y = Cx + Du \tag{35b}$$

To calculate the LQR gain matrix `K`, the default `K = lqr(A,B,Q,R)` MATLAB function is used which takes as input the steady-state model matrices A and B as well as weighting matrices Q and R. Matrix multiplication then occurs between the 16x1 state error vector and the 4x16 LQR matrix `K` to obtain the 4 input voltages `u` to the system. These are then "artificially"

saturated by a block that sets the upper and lower voltage limits prior to passing on to the model. This wording is chosen because the LQR controller does not "know" that the model will not receive the voltages that are too high or too low. The outputted quadcopter states are then fed into the Trim State bias block which adds to the last four states (rotor velocities ω_{Ri}) the velocity required to keep the quadcopter at hover. These states are then separated by a MATLAB function block into multiple scopes for the monitoring and recording of data.

A default LQR matrix is initially created where Q and R are diagonal unitary matrices, placing equal emphasis on actuator effort and quadcopter state error. This matrix is then optimized to more strongly penalise positional error in the earth frame (states x, y and z) and is more generous with actuator effort. A unit step input to all three displacement states is applied to the reference signal at time $t = 1$ s. This scenario will amplify any potential differences in the performances of each controller as it requires accurate and varied inputs to each rotor to quickly and efficiently minimise the error in the quadcopter states. The results are shown in Fig. 7.



(a) Default LQR Matrix Applied

(b) Optimised LQR Matrix Applied

Figure 7: Linear Model Response to LQR Control (Reference: $x = y = z = 1$)

In Fig. 7a, the quadcopter reaches and stabilises at the reference position in the x- and y-axes approximately 5 seconds after the step input is applied. However, the reference altitude is not yet reached, and will likely take another 10 seconds of simulation time to reach the reference $z = 1$. This is most likely due to the LQR controller balancing the error in the rotor velocities and the error in the z position, reluctant to apply too much voltage to the rotors at once. However, the optimised LQR controller used in Fig. 7b shows an improved system response. Despite an initial overshoot of 11%, the quadcopter stabilises $\approx 20\%$ faster in the x and y directions and $\approx 80\%$ faster in the z direction. The optimised LQR controller employs a more liberal use of the actuators to reduce the all-important positional error of the quadcopter states. It is to this version of the controller that future controllers discussed in this section will be compared to.

4.2 MPC

The LQR controller is now replaced by an MPC controller. This is done by replacing the Sum block and the `K` gain block with the MPC block readily available within MATLAB's Model Predictive Control Toolbox. The system diagram resulting from this is shown in Fig. 8.

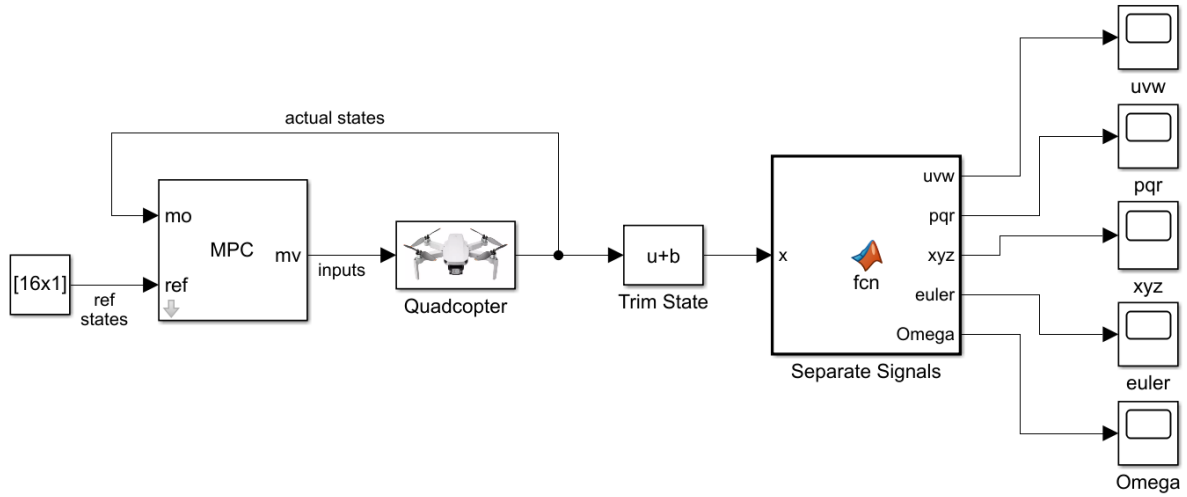
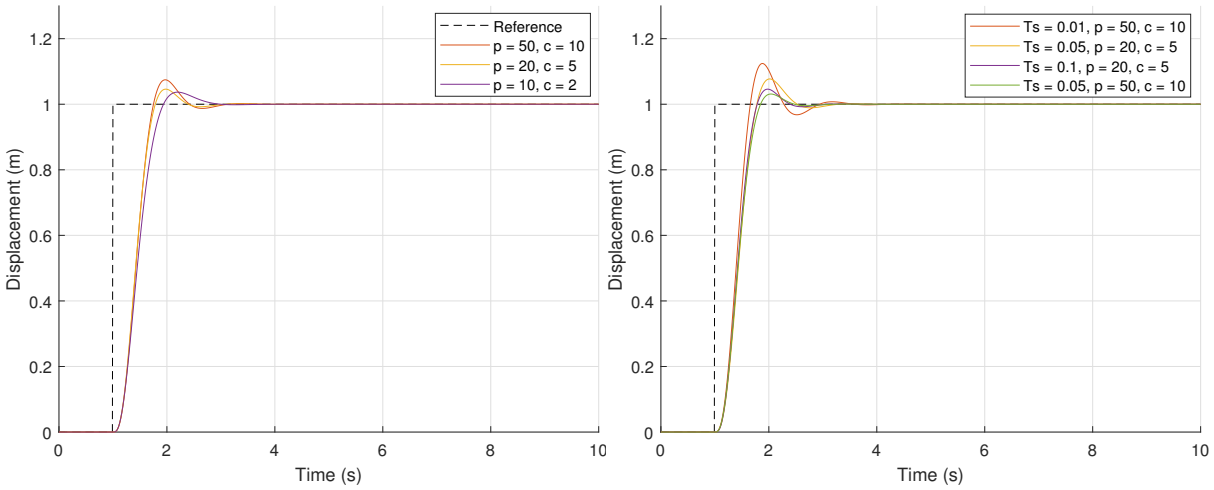


Figure 8: Diagram of Linear Quadcopter Model with MPC

The MPC controller is tuned within the MPC block. The key controller properties to adjust initially are the sample time `Ts`, the prediction horizon `p`, and the control horizon `c` (see section 2.3). However, the first element to edit is the "default scenario" within the MPC Designer app, which sets a step input of 1 at $t = 1$ to all states and optimises the system inputs based on that. This is changed to a constant (nominal value) for all states except z, where the step input is kept. System response under a range of configurations is shown in Fig. 9, with the legend generally in order of least favourable response (top) to most favourable (bottom).



(a) Varying Horizons

(b) Varying time steps and horizons

Figure 9: Effect of sample time Ts, Prediction Horizon p, and Control Horizon c on system response

Note that the system response is unstable for Ts = 0.01, p = 20, and c = 5, and therefore there are two different sets of prediction and control horizons used across the 0.1-0.01 time step range. This is because the system could only "see" 0.2 seconds ahead, not far enough to see the reaching of and stabilisation at the reference altitude, leading to highly erratic behaviour. The optimal balance between accuracy and computational effort was found to be `Ts` = 0.1, `p` = 20, and `c` = 5.

Using MATLAB's "slider feature" (see Fig. 10), one can change how aggressive the MPC

controller's inputs are. This slider adjusts the ratio between the positional error weight and the MVs (Manipulated Variables) error weight, with "aggressive" performance tuning placing a large weight on positional error and a lesser weight on MVs (actuator effort). The MVs are the 4 rotor voltages fed to the quadcopter model which then outputs the 16 states (Measured Outputs, or "MOs") as a result. A "robust" closed-loop performance will proceed cautiously, valuing stability over settling time, with a larger penalty on actuator effort and a lower penalty on positional error. In Fig. 11, the positional error weight is kept at 10, and the MV weight is varied between 1 and 0.

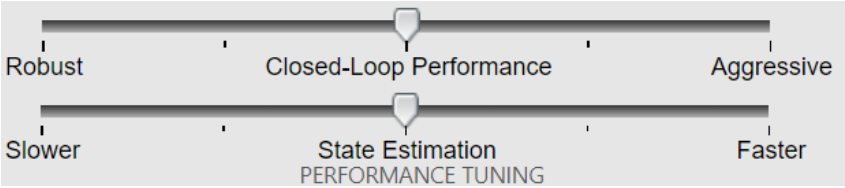


Figure 10: MPC Designer - Performance Tuning Sliders

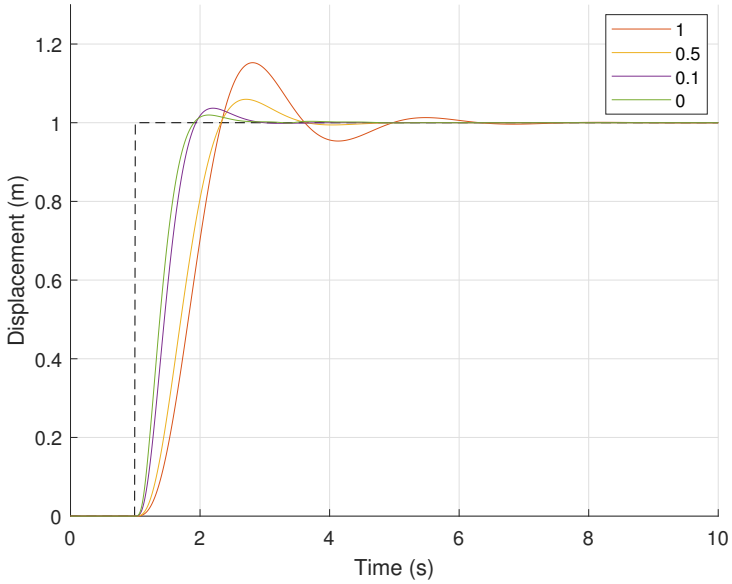
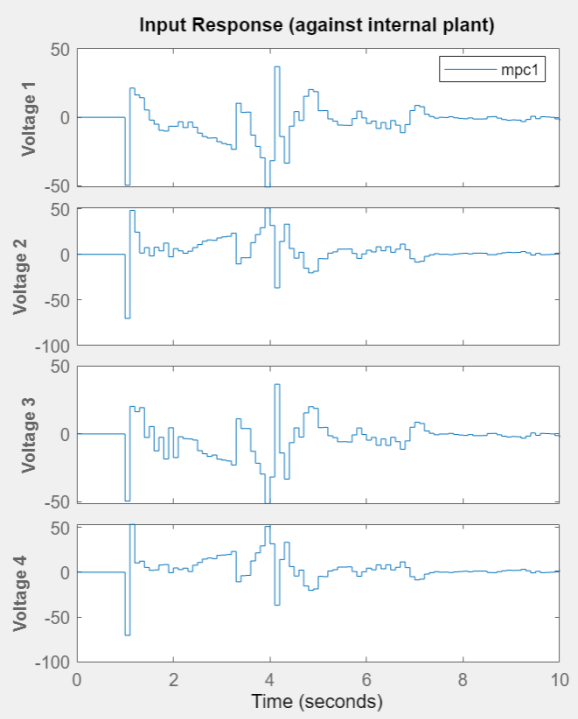


Figure 11: Effect of changing MV weight on system response

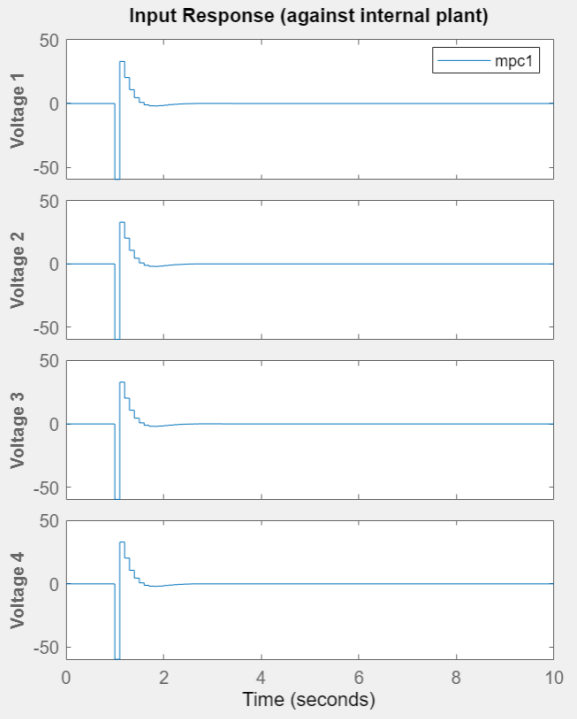
The maximal aggression applies a weight of 10 to the position states x, y and z, and 0.1 to the MVs. Giving the MVs a weighting means that the controller will be penalised for not returning to the nominal value of 0 once the state errors have been eliminated. A weight of 0 is applied to the 13 other states and to the MV rates, meaning that the controller can employ aggressive voltage changes as freely as possible to reach reference positions. However, when an MV weight is omitted, it can result in input noise (see Fig. 12).

Although both systems generate similar responses (see Fig. 11), giving the MV error a non-zero weight results in more stable quadcopter behaviour. This ratio of 0.01:10 for the MV:positional error weight will be kept in the MPC controller hereinafter.

Constraints are then added to the controller so that its outputs, or "Manipulated Variables" (MVs) are within the bounds acceptable by the motors. These values are the same as the limits imposed on the LQR saturation block, that is 8.624 and -6.176. The same step input reference scenario that was used in the previous section in Fig. 7 is used in Fig. 13, with the default MPC controller shown in Fig. 13a and the fully tuned controller shown in Fig. 13b.

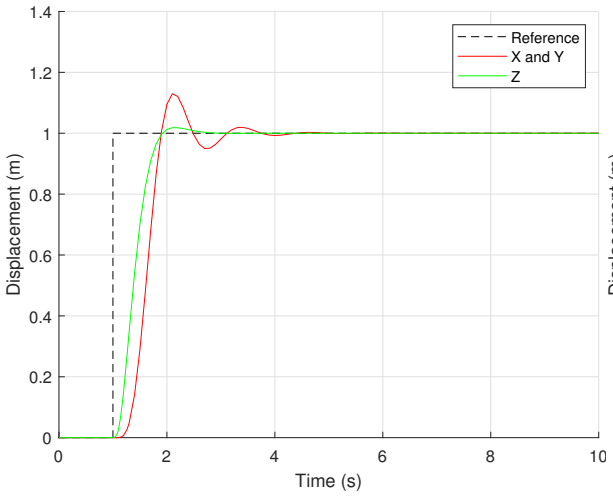


(a) MV weight set to 0

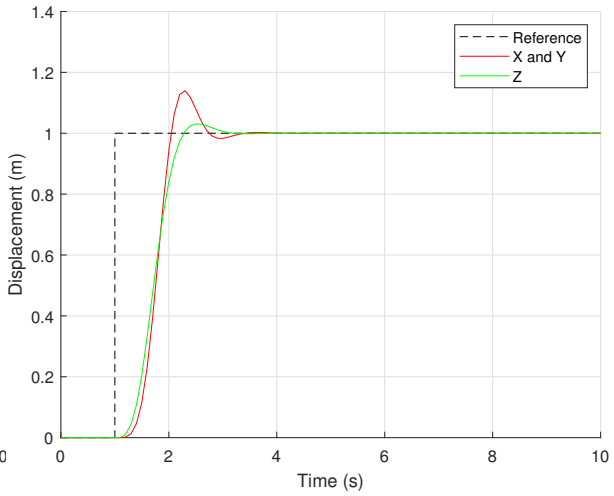


(b) MV weight set to 0.01

Figure 12: Rotor Input Voltages for different MV weights



(a) MPC without constraints



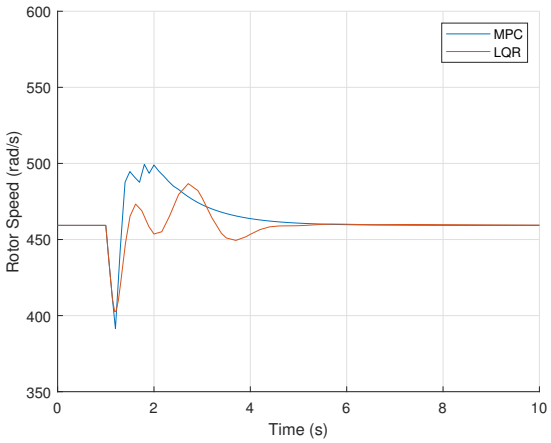
(b) MPC with constraints

Figure 13: Linear Model Response to MPC Control (Reference: $x = y = z = 1$)

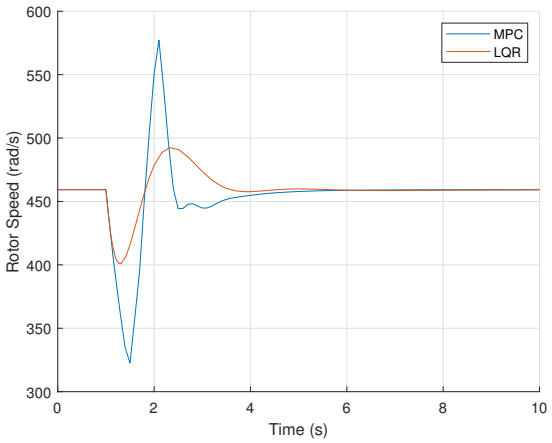
4.3 Comparison of LQR and MPC

Although it is interesting to note that the default MPC controller provides much better performance the default LQR controller, it is the comparison between the tuned versions of the controllers that is important. The MPC controller seems to stabilise in x, y and z at around the 3-second mark, resulting in a 33% increase in performance. The reason for this can be seen in the superimposition of the rotor speeds under each controller method (see Fig. 14).

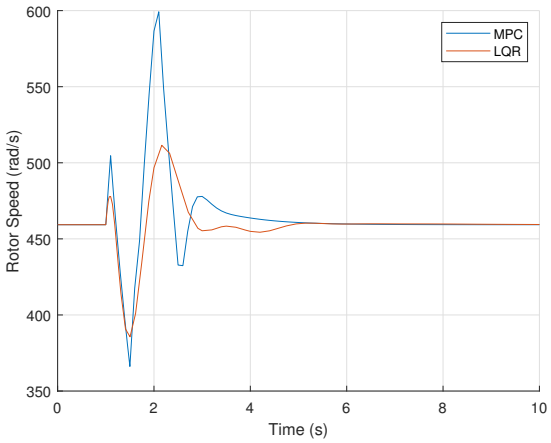
The MPC controller is clearly able to reach more extreme speeds, both above and below the



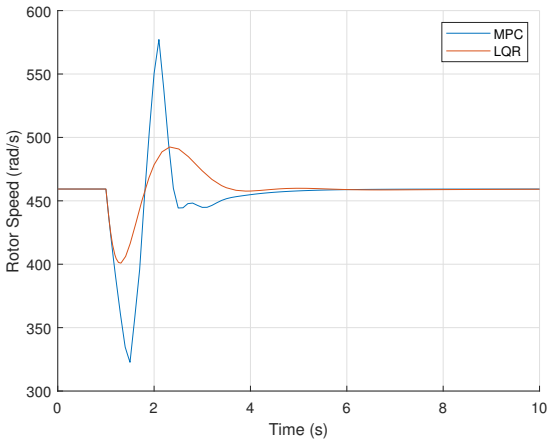
(a) Rotor 1 Speed



(b) Rotor 2 Speed



(c) Rotor 3 Speed

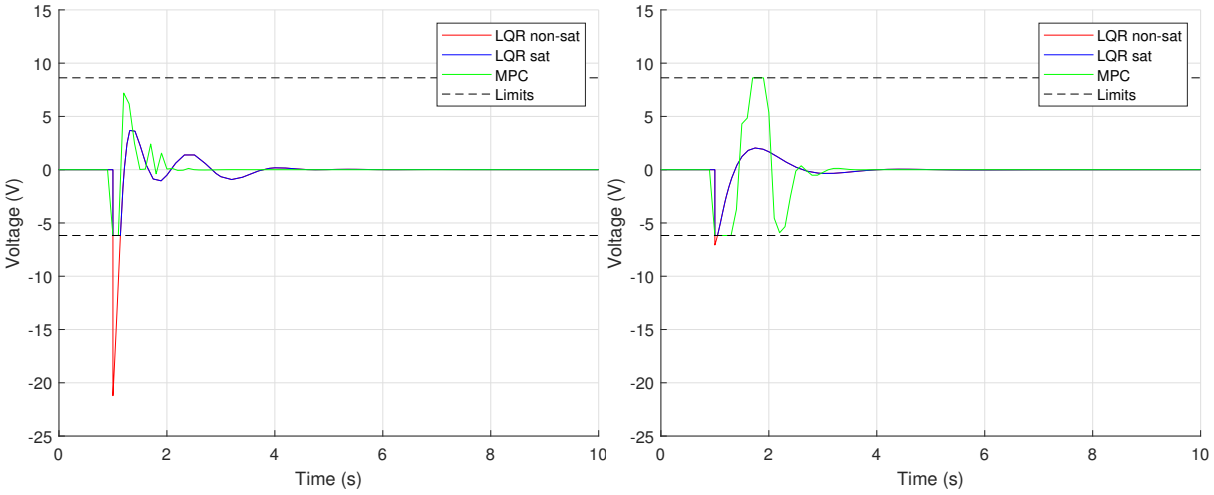


(d) Rotor 4 Speed

Figure 14: Rotor Speeds for $x = y = z = 1$

nominal value of 459.3, compared to the LQR controller. This is a result of the effect of saturation on the input signal, as evidenced in Fig. 15. It can be seen that the LQR controller applies a sharp negative spike as a control input to rotor 1 as soon as the reference signal is applied, expecting the model to drastically reduce velocity. However, the signal is saturated and as such the rotor's speed does not decrease as much as the controller expected it to and as a result of this all too brief command, the rotor does not fully settle to a lower velocity before the next control signal is issued by the LQR controller to re-increase the speed. Conversely, the MPC controller benefits from the model taking the inputs directly issued by the controller as the constraints are built-in. It is therefore "aware" of the fact that it must issue this signal at its lower voltage limit for a duration of multiple time steps to achieve the desired lowered rotor speed. This combined with the fact that it can predict the states of the system up to 20 time steps (2s) away means that it can hold the voltage at a lower value for longer (see Fig. 15b) knowing it will still be able to stabilise 2 seconds later at $x = y = z = 1$. Note that the voltage values displayed in Fig. 15 exclude the base voltage required to maintain hover, and are only representative of the change in voltage from a stable hovering state. The optimisation-led approach of the MPC also results in increased computational time, from 0.49 seconds to 0.59 seconds (averaged over 50 simulations).

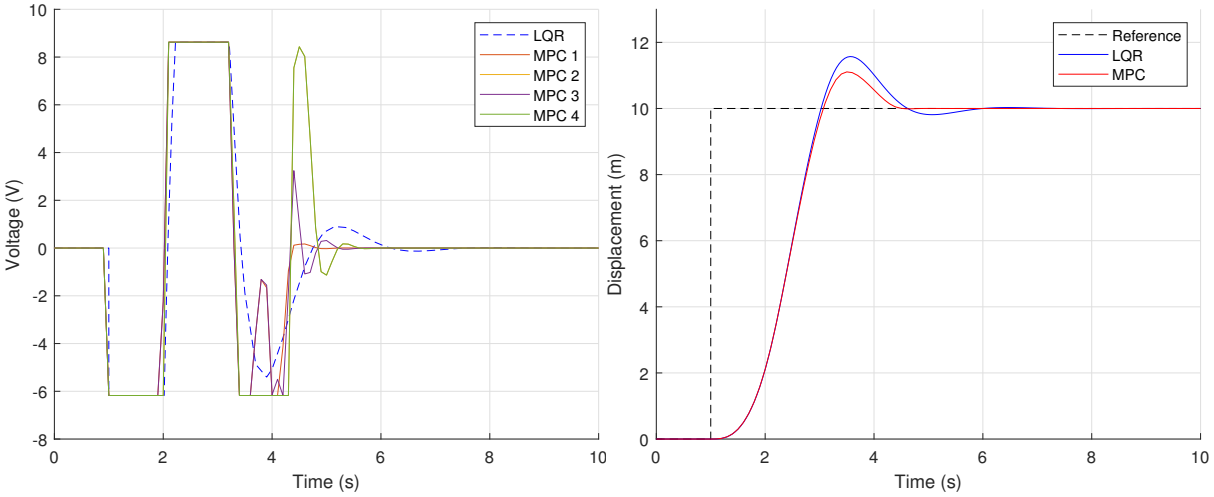
To amplify the difference between the linear quadcopter model's response to LQR and MPC control, the reference signal is increased to $z = 10$. The results are displayed in Fig. 16, with



(a) Rotor 1 (b) Rotor 2

Figure 15: Inputs to rotors 1 & 2 from LQR (pre- and post-saturation) and from MPC Control

the control signal inputs on the left and the drone position in z on the right.



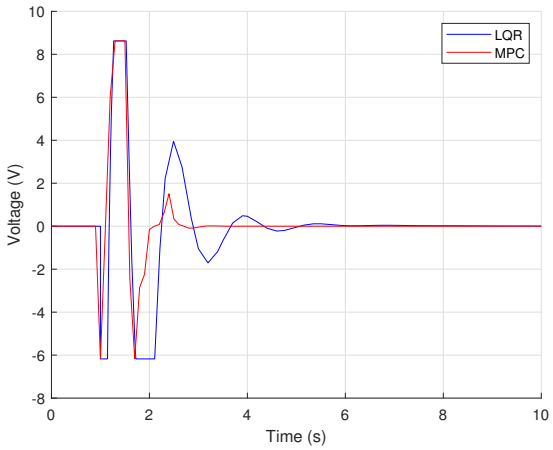
(a) Control Signal Input to Linear Model (b) Drone Position in Z

Figure 16: MPC vs LQR for z = 10

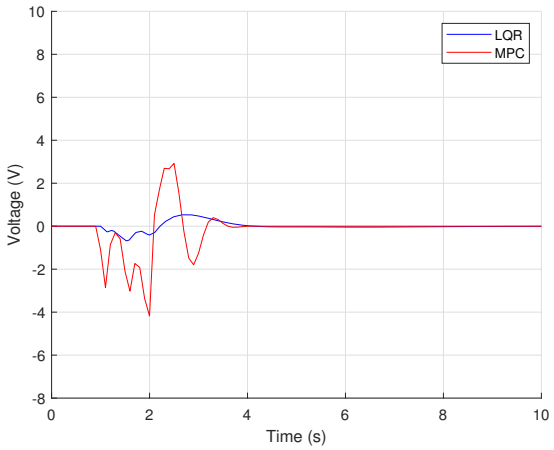
The LQR controller applies the same voltage to each motor at every moment in the simulation, and is therefore represented by a single signal in Fig. 16a. Conversely, the MPC controller can adjust each rotor individually and has therefore more control over the rotational forces within the quadcopter. As seen in Fig. 16b, the MPC displays slightly better performance, illustrated by a reduced overshoot as well as an earlier and smoother stabilisation at $t = 4.3$ s, as opposed to $t = 6$ s for the LQR. However, it uses more power as the signal input is spread across a wider range of the voltage spectrum in Fig. 16a.

A horizontal manoeuvre is demanded of the system next to further amplify the difference between control methods. In this scenario, the reference signal $x = y = 2$ is applied to the system at $t = 1$ s. The control signal inputs for each rotor are displayed in Fig. 17 and the drone position in x and y in Fig. 18.

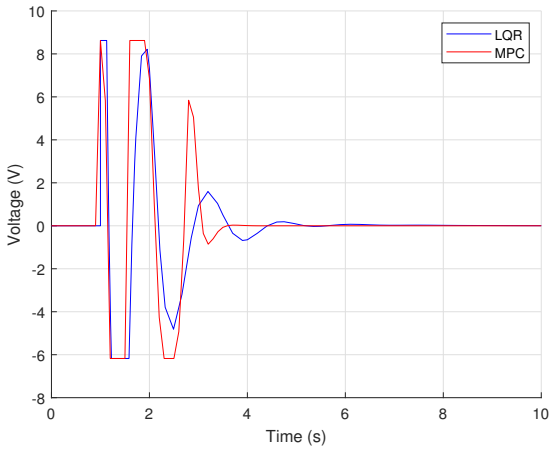
The LQR controller intends to quickly lower the first rotor's velocity to initiate a dip in the



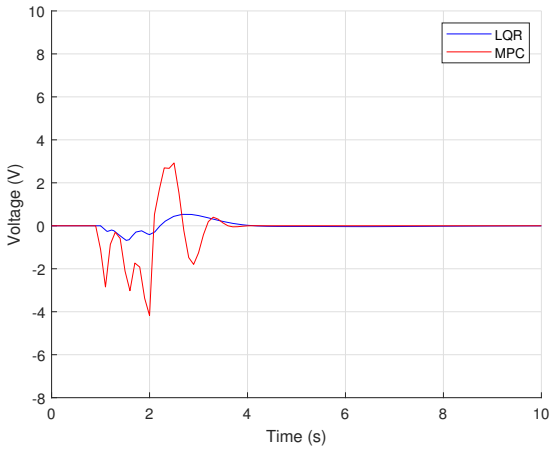
(a) Rotor 1 Voltage



(b) Rotor 2 Voltage



(c) Rotor 3 Voltage



(d) Rotor 4 Voltage

Figure 17: Linear Model Control Signal Input for $x = y = 2$

$+x, +y$ direction and simulatenously increase the velocity of rotor 3 at the same rate, whilst barely making use of rotors normal to the intended direction of travel (2 and 4). Contrarily, the MPC knows that only adjusting the velocities of rotors 1 and 3 will not result in the quadcopter reaching the reference position quickly enough, and therefore uses rotors 2 and 3 to accelerate the drone in the desired direction once it has dipped towards $+x, +y$. The benefits of this approach are made clear in Fig. 18 below, where the MPC controller settles 50% faster than the LQR, and with negligible undershoot.

In fact, the system response under LQR control displayed in Fig. 18 hints at the reason for evaluating such a modest translation across the XY plane: the system becomes unstable for values greater than $x = y = 2.2$ (see Fig. 19). It is hypothesised that the reason for this instability is the linearisation of the model around the point of hover, which does not translate well to horizontal motion. Once the system reaches a certain velocity in the XY plane, the controller cannot stabilise the system in time and oscillates between voltage limits as the quadcopter flies helplessly on in the $+x, +y$ direction. This theory is pursued further in section 4.5.

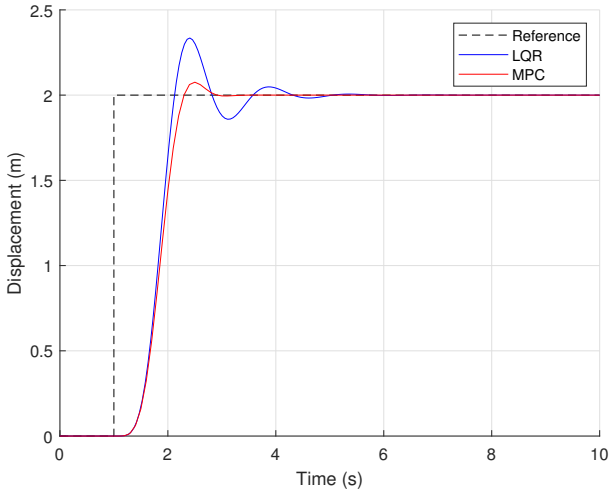


Figure 18: Drone Position in x and y under LQR and MPC control

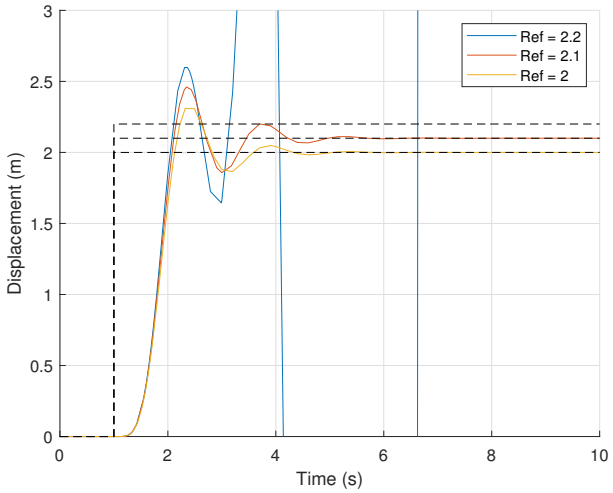


Figure 19: LQR linear model instability for increasing reference values of x and y

4.4 Multiple MPC

As expected, the MPC controller was proven to have superior performance over the LQR controller for a wide range of simulations. The next task was to make this controller robust, such that if a rotor failure (or two) were to occur mid-flight, the MPC controller would automatically compensate for this and maintain reference states. This is done by using MATLAB's Multiple MPC block, also readily available in the Model Predictive Control Toolbox. Separate MPC controllers are designed for each operating condition, or fail case, and depending on the system's state, the multiple MPC controller switches between controllers mid-simulation and keep the quadcopter at the reference position.

For this, a total of 7 MPC controllers are created: 1 for standard operating conditions, where all rotors are fully functioning, 4 "single-failure" controllers, one for each case where a certain rotor undergoes full failure, and 2 for "double-failure" cases, where rotors 1 and 3 or rotors 2 and 4 have both fully failed. It has been shown that due to quadcopter dynamics it is possible to maintain level altitude and attitude if 2 rotors across the CG from each other fail. Maintaining safe flight in the case of 2 adjacent rotors or 3 or more rotors failing is unreliable [16], and in this case the Multiple MPC block would switch to the most accurate controller and attempt an

emergency landing by slowly lowering in altitude.

Each failure MPC controller is configured such that the minimum and maximum value for the voltage of the relevant rotor is set to 0; it therefore can only influence the system by controlling the 2 or 3 other rotors.

The Multiple MPC block takes as input the quadcopter states, the reference states, and a switch signal. It outputs the optimised control signal inputs (MVs), as well as an optional output, the estimated states. Two separate avenues are taken for the implementation of this multiple MPC controller. The first is a logic circuit that identifies which (if any) of the 4 MVs are 0, and relays a number to the "switch" input on the multiple MPC block. This worked well for a low amount of MPC controllers, however as additional rotor-failure controllers were included, the logic circuit quickly became convoluted.

Instead, the "estimated states" optional output of the MPC controller block is utilised (see Fig. 20). This option outputs the estimated controller states for the rest of the prediction horizon. The 7 MPC controllers are stacked in parallel, all receiving the same inputs (the quadcopter states and the reference states), and outputting different MVs and estimated states. A MATLAB function is then used to pick the best controller for the next time step. The logic behind this is illustrated by a programming flowchart in Fig. 21.

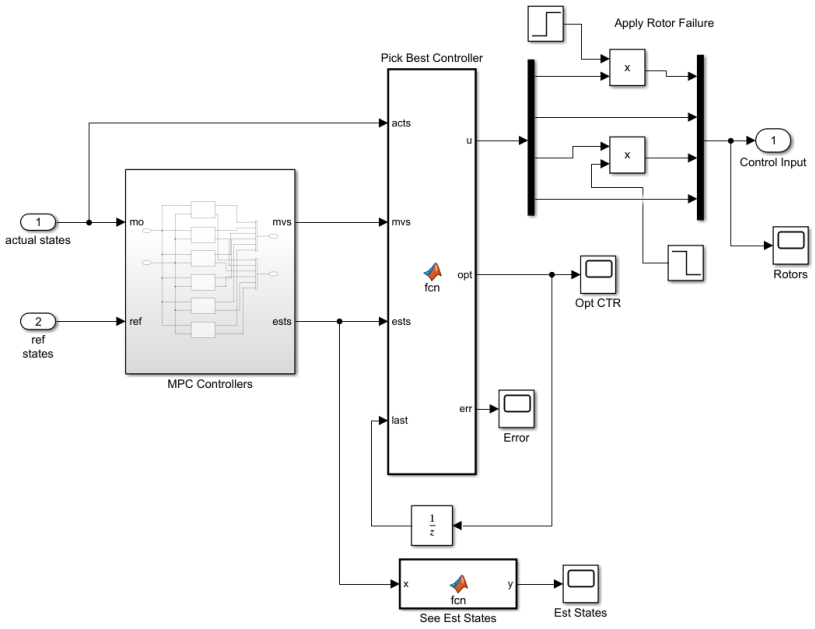


Figure 20: Multiple MPC Selector

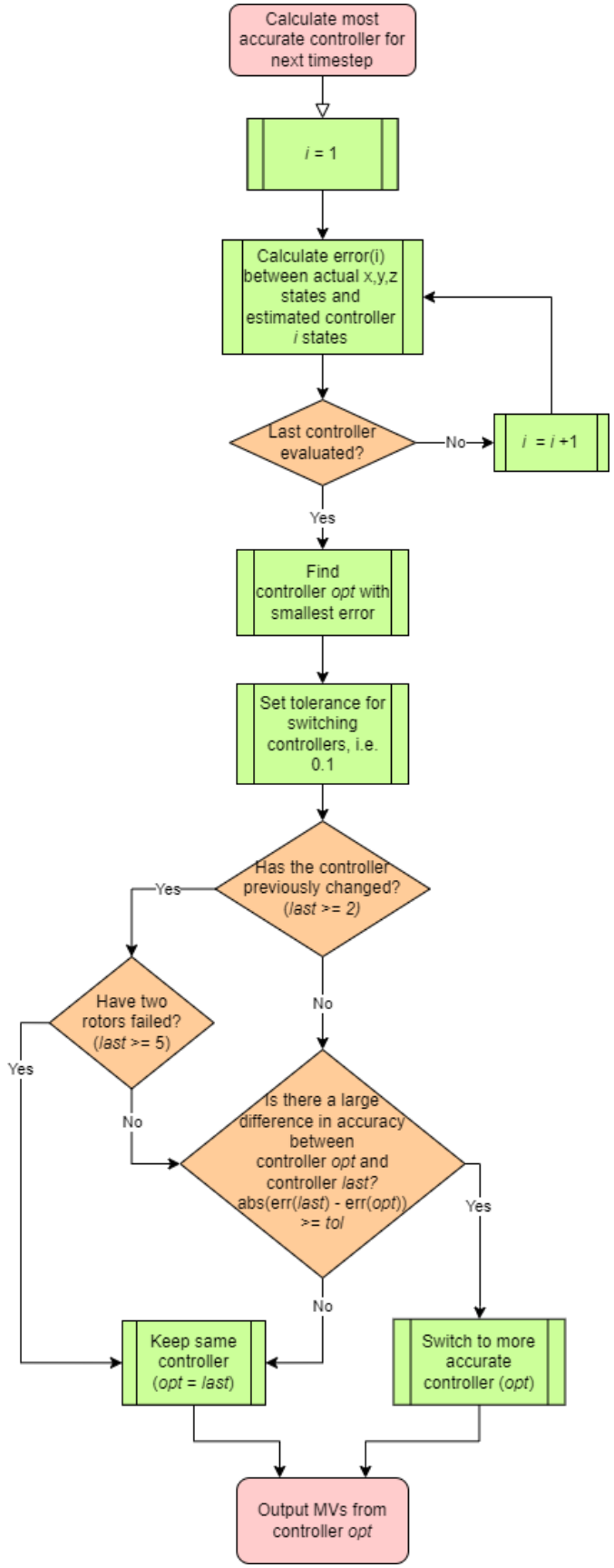


Figure 21: Pick Best Controller

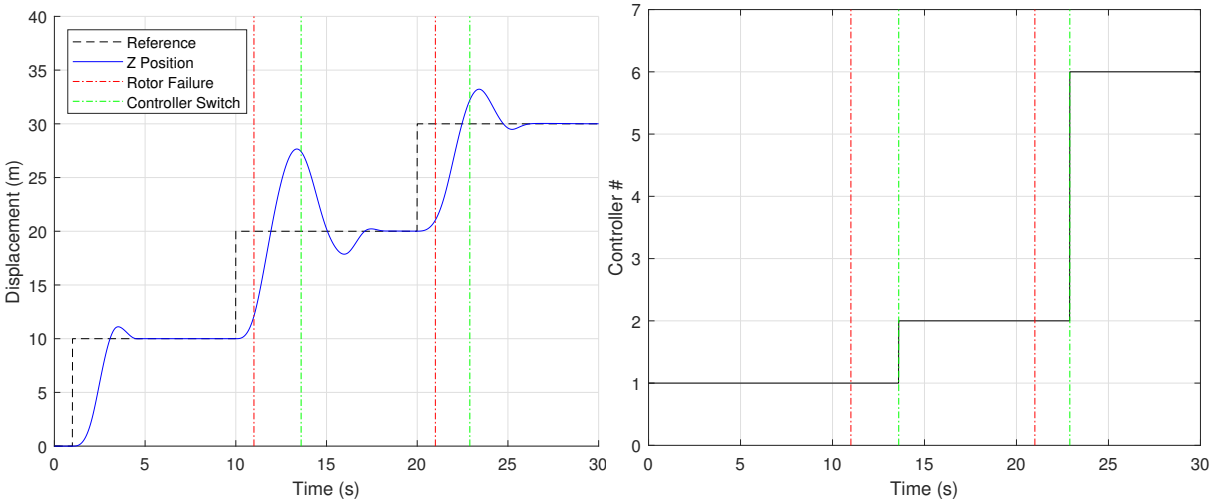
For the controller switching to occur, the quadcopter must be mid-manoeuvre, as the steady

hover state is equivalent to all rotor voltages at 0. In this case, the estimated states from all the controllers would be 0, and they would all be outputting a constant zero signal across the 4 MVs. Thus, the implementation of a rotor failure must occur whilst the quadcopter is moving between reference positions, as there will be discrepancies between the estimated states of each controller. A reference signal is therefore designed with three large step changes in the z position, to be able to test multiple rotor failures within one simulation:

- At $t = 0$, $z = 0$.
- At $t = 1$, $z = 10$.
- At $t = 10$, $z = 20$.
- At $t = 11$, rotor 1 fails.
- At $t = 20$, $z = 30$.
- At $t = 21$, rotor 3 fails.

The rotor fault is applied by separating the 4x1 MV signal into separate rotor voltages and multiplying one of them by a reverse step input, i.e. a block whose value steps down from 1 to 0 at a certain time step. The most accurate MPC controller at that time is therefore the one which is unaffected by the nullifying step signal, resulting in a controller switch.

It is assumed in the logic of the controller switching function and therefore in the context of this simulation that once a rotor fails, it will remain in a failed state until the end of the simulation. This hypothesis is in line with most real-world, practical situations [34]. Fig. 22a shows the altitude of the quadcopter as well as the exact time steps where rotor failure occurs, and Fig. 22b shows the active controller at each time step.



(a) Quadcopter z position

(b) Active Controller

Figure 22: Active controller

During the testing of this situation, it was noticed that the time step (kept at 0.01 s) and the error tolerance were both crucial to obtaining the intended result consisting of the controller switching shortly after each rotor failure. The fine-tuning of this threshold that had to be crossed by the difference between the current and optimal controller estimated states decided

when, if ever, the active controller would switch. If the tolerance was set too low (or altogether omitted as was the case in earlier versions of the algorithm), the approximation of the ODE solver and rounding errors led to premature switching between controllers and an unstable response. Conversely, a tolerance set too high saw the controller switching very late, in some cases not at all.

As evidenced by the figure, the error tolerance was crossed 2.6 s and 1.9 s after the failure occurred. This is likely due to the fact that during these increases in altitude, all the controllers were outputting maximum voltage to the rotors, and that therefore switching was not necessary. It is only once the quadcopter reversed outputs that the controllers started calculating different estimated states, at which point a switch had to occur. This theory is corroborated by a similar simulation where the reference signal steps in the negative direction (representative of an emergency landing situation).

The overshoot and undershoot from each step input is also very telling, the largest of which occurs during the second step input. The quadcopter provides better tracking during the third step input, with the first step input serving as the example of an optimal response. To prove that the controller switching method is both functional and relevant, the same reference signal and rotor failure points are applied to a system without Multiple-MPC functionality. The results are shown in Fig. 23.

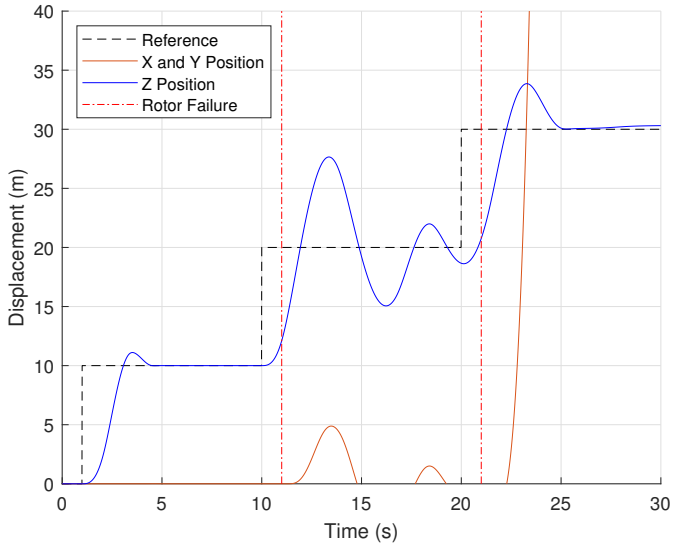


Figure 23: Reference Signal from Fig. 22 without controller switching

The MPC controller that believes all rotors to be working demonstrates considerably worse performance in altitude adjustment after the second step input, not fully stabilised by the moment the third step input is applied. It is also worth noting that the x and y positions oscillate between +5 and -5 (superimposed as they are identical at each time step). After the third step input is applied, the quadcopter appears to overshoot and stabilise at $z = 30$ remarkably quickly. However, the quadcopter is not stable as its position in x and y increases exponentially from this point on. This is what renders the quadcopter unstable and unable to regain its reference position, as its altitude z continues to increase from $t = 25$ onwards.

It is equally important to study partial failures, as some quadcopters consist of two rotors on each arm [35]. Rotor faults in systems equipped with this type of redundancy can be modelled as partial failures, such as a 50% decrease in voltage. Once again, the reference signal is applied to a single MPC system, however with a 0.5 factor to the relevant rotor voltage, rather than

0. This is done to evaluate whether additional controllers need to be designed around partial failures to improve performance in this type of operating condition. The results are shown in Fig. 24.

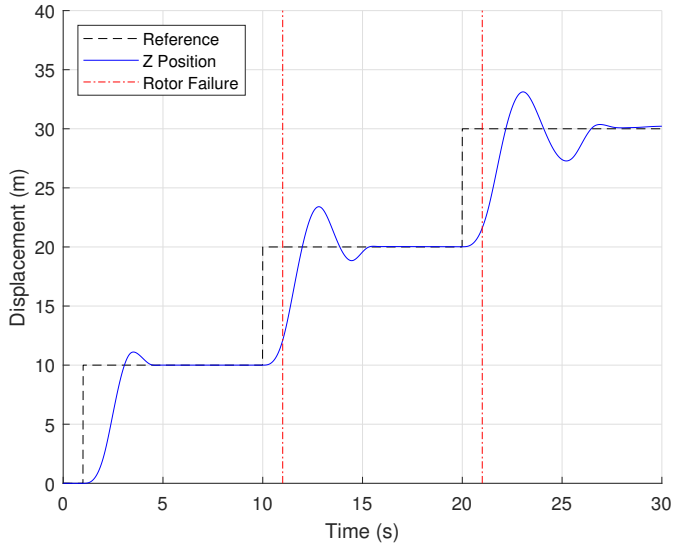


Figure 24: Reference Signal from Fig. 24 with partial rotor failures on single MPC

As evidenced by the figure, the results are relatively satisfactory. The third step input results in slightly more oscillation on the part of the quadcopter, however the considerable increase in computational expense tied to the multiple MPC is considerable. The single-MPC simulation shown in Fig. 24 takes on average 0.64 seconds to complete, whereas the slightly more accurate multiple MPC version takes 2.44 seconds, a 4-fold increase.

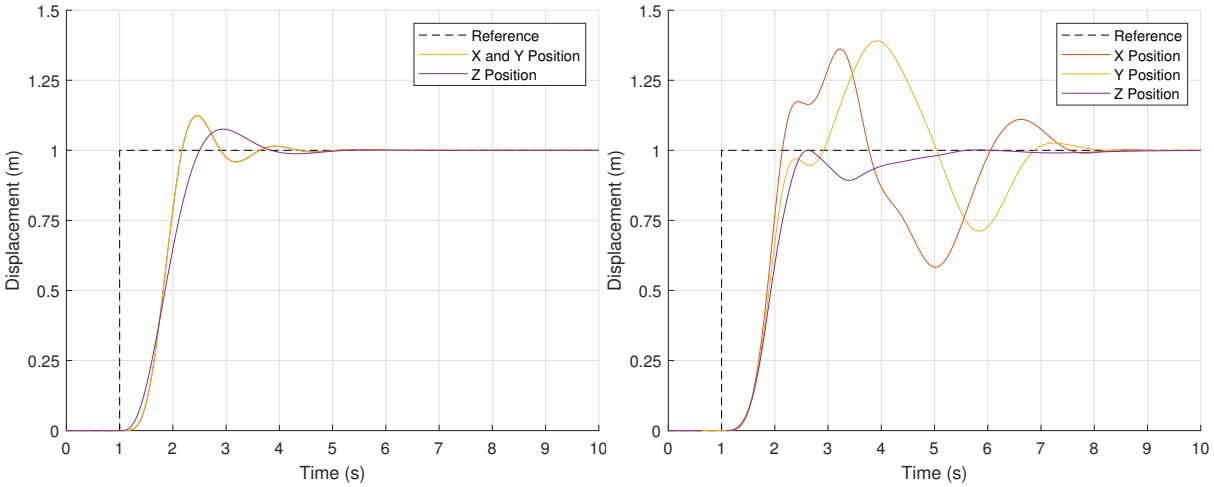
It can therefore be said that multiple MPC is a reliable way of managing a mid-flight rotor failure. However, if possible, the other onboard controllers should be left dormant until a fault is detected. Once this happens, the 7 controllers described at the start of this subsection should be activated with their respective estimated states evaluated to find the most optimal controllers. Additional controllers designed off of various partial rotor failure conditions are not necessary as the default MPC has been shown to provide adequate performance in this case. This should be reflected by a switching tolerance high enough to avoid switching in the case of partial failure but low enough to switch soon after full rotor failure.

This experiment also demonstrates the importance of fault detection within a system. A robust quadcopter must be able to identify faults as soon as they happen and reconfigure the impacted areas, such as the controller, in order to maintain flight operations [11].

4.5 Non-linear Model

The quest began to implement a non-linear MPC controller into the non-linear model. As mentioned in section 3.10, the linear model is essentially a snapshot of the non-linear model, with the dynamics relating to the trim (hover) condition being extended to all other operating conditions. The non-linear model takes into account a much wider range of flight dynamics, and is therefore expected to take much longer to optimise for in the case of the MPC controller. Additionally, despite the LQR's much quicker response (in the same order of magnitude as the linear simulation as a simple matrix multiplication is needed to get from state errors to plant

input), it also struggles much more to stablize the system (see Fig. 25).



(a) Linear Model (b) Nonlinear Model

Figure 25: LQR controller performance for $x = y = z = 1$

As shown in the above figure, the nonlinear model is much more unstable, especially when moving in the x and y axes. Any simulations with $x_{ref} > 1.2$ or $y_{ref} > 1.2$ were completely unstable with the LQR controller. A better approach is thus needed to control the quadcopter in the 3 cartesian axes.

To facilitate the simulation and debugging of the NLMPC, the nonlinear model (essently a Simulink-based programming block function) was translated into a single MATLAB function, using linear code rather than blocks to manipulate values. This is the "State Function", the state derivative function that is discretised using the implicit trapezoidal rule. It is mentioned in the MATLAB documentation that its prediction accuracy is highly dependent on the controller sample time `Ts`, and that inaccurate values are likely to be obtained if this time step is set too high. This translation step was not foreseen and was considerably time-consuming, however the risks and contingencies that were laid out in the Project Scoping and Planning report for this FYP accounted for such unforeseen tasks and therefore time was found to perform this task. Rather than simply running the simulation in Simulink with an automatic solver, the `nlmpcmove` function was used, which takes as input the nonlinear MPC controller object `nlmpc` (a structure containing various controller settings), the previous quadcopter states `xk`, the previous plant inputs `lastMV`, the reference states `yref`, and a solution details structure `nloptions` (containing optimal values across a prediction horizon to be used as initial guesses for future optimisations in upcoming time steps, the number of iterations, and the objective function cost, among other details).

To validate the new MATLAB function, the nonlinear model was simulated with the LQR controller and a fixed time step of 0.01. For a 10-second simulation, this results in 1001 combinations of inputs. The reference signal was $z = 1$. These plant inputs were then captured and fed into the nonlinear state function `nlStateFcn.m`, which is in the form `xout = nlStateFcn(xin,uin)` (in essence, the prediction model takes the previous set of states as well as the current voltage inputs to output a new set of states). The pseudocode for this validation script is shown in Algorithm 1 below, with comments across from each line explaining what the step is doing.

The results for a range of solver time steps are shown in Fig. 26, and the timings for various ODEs in Table 3.

Algorithm 1 Validate nonlinear state prediction model

for $k = 1 : Duration/Ts$ **do** ▷ For each time step
$xk \leftarrow xHistory(k,:)$ ▷ Compute the control moves with reference previewing
$uk \leftarrow MVs(k,:)$ ▷ Store voltage data from relevant time step for next solve
$uHistory(k+1,:) \leftarrow uk'$ ▷ Store voltage data in voltage history matrix
$ODEFUN \leftarrow nlStateFcn$ ▷ Set state function as the ODE to be solved
$YOUT \leftarrow$ `ode45`$(ODEFUN, [0\ Ts], ...$
$xHistory(k,:)')$ ▷ Solve ODE using `ode45` solver for init. conds. [`0 Ts`] across $xHistory$
$xHistory(k+1,:) \leftarrow YOUT(end,:)$ ▷ Save last row of (converged) states resulting from ODE solving
end for

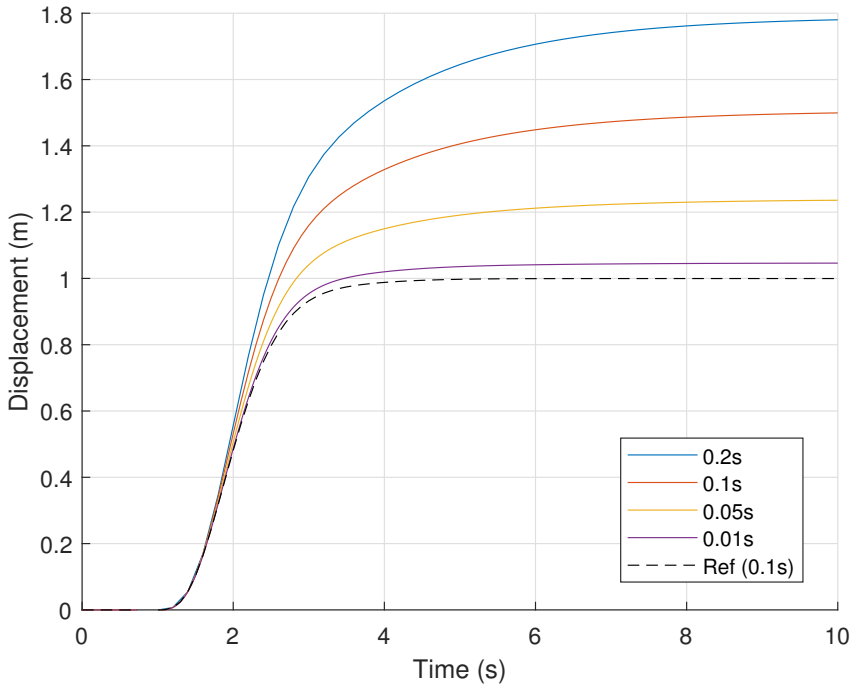


Figure 26: Sample time `Ts` effect on prediction model accuracy

As can be seen in Fig. 26, the only time step that is able to closely follow the reference position is `Ts = 0.01`. This is due to the intrinsic nature of the prediction model and the MATLAB function used to optimise it, which is seemingly less accurate than the Simulink solver (`ode3`). Therefore, for future NLMPC simulation, a sample time of 0.01 must be used. An even smaller sample time could theoretically be used, however the computational time is likely to already be much higher than the linear model, thus this research avenue will be reserved for later on.

Table 3: MATLAB ODE solver times

Solver	Accuracy	Comp. Time (s)
`ode23`	Low	9.52
`ode113`	Low to High	6.91
`ode45`	Medium	18.03
`ode78`	High	49.47
`ode89`	High	61.49

Although the computational time for various ODE solvers varies wildly as shown in Table 3, the resulting states are identical across all 101 time steps. The listed accuracies come from

MATLAB documentation which describes when to use which solver. As no difference in the solution can be found between solvers, the default solver `ode45` will be used hereinafter (the MATLAB documentation also specifying that this is the first solver that one should try).

With the nonlinear prediction model validated for a time step of ≤ 0.01, the next step was to have the NLMPC controller solve the $z = 1$ problem on its own. The pseudocode for doing this is shown in Algorithm 2.

Algorithm 2 Run NLMPC on nonlinear model

$nx = 16$ ▷ Specify model settings
$ny = 16$
$nu = 4$
$nlobj \leftarrow \texttt{nlmpc}(nx, ny, nu)$
$nlobj.Model.StateFcn \leftarrow nlStateFcn$

$nlobj.Weights.OutputVariables \leftarrow zeros(1, 16)$ ▷ Set weights
$nlobj.Weights.OutputVariables(7:9) \leftarrow 10$
$nlobj.Weights.ManipulatedVariables(1:4) \leftarrow .1$
$nlobj.Weights.ManipulatedVariablesRate(1:4) \leftarrow 0$

$Ts \leftarrow .01$ ▷ Specify NLMPC options
$p \leftarrow 200$
$c \leftarrow 50$
$nlobj.Ts \leftarrow Ts$
$nlobj.PredictionHorizon \leftarrow p$
$nlobj.ControlHorizon \leftarrow c$
$nlobj.MV.min \leftarrow -6.176$
$nlobj.MV.max \leftarrow 8.624$

$nloptions \leftarrow nlmpcmoveopt$ ▷ Create solution info structure
$nloptions.MVTarget \leftarrow [0000]$ ▷ Nominal control to keep the quadcopter floating
$mv \leftarrow nloptions.MVTarget$
$Duration \leftarrow 5$
$time \leftarrow zeros(Duration/Ts, 1)$
$xHistory \leftarrow zeros(1, 16)$ ▷ Initialise history matrices
$xHistory(13:16) \leftarrow 459.3008$
$infoHistory \leftarrow zeros(Duration/Ts, 3)$
$lastMV \leftarrow mv$
$uHistory \leftarrow lastMV$
$yref \leftarrow zeros(p, 16)$ ▷ Set reference signal for z and ω_r
$yref(:, 9) \leftarrow 1$
$yref(:, 13:16) \leftarrow 459.3008$

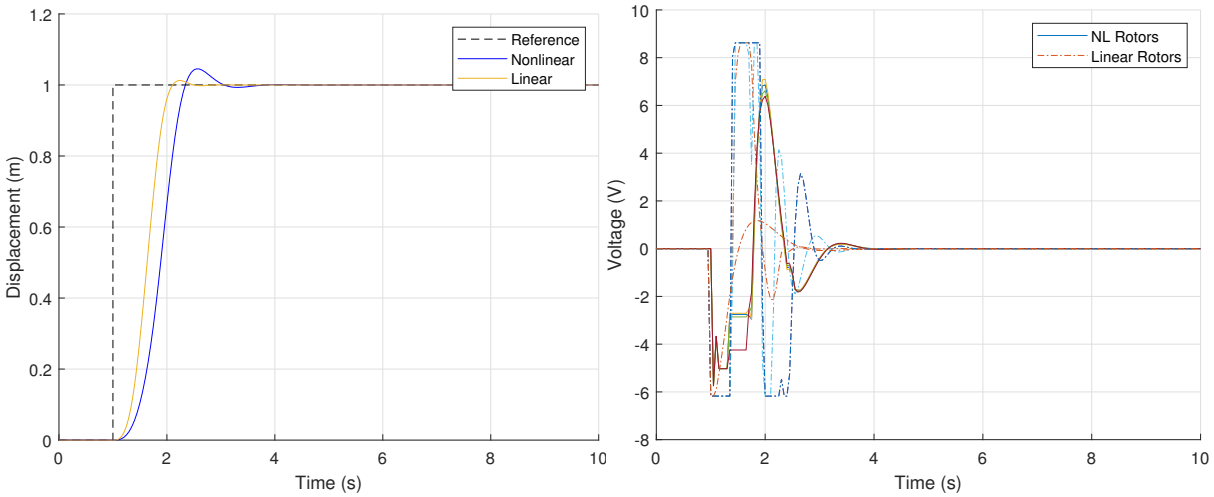
for $k = 1 : Duration/Ts$ **do** ▷ Begin main simulation loop
  for $i = 1 : p$ **do** ▷ Set initial guesses across prediction horizon
    if $i + k - 1 <= length(MVs)$ **then**
      $nloptions.MV0(i,:) \leftarrow MVs(i + k - 1,:)$
    else ▷ If no more guesses, set rest to 0
      $nloptions.MV0(i : p,:) \leftarrow 0$
      `break`
    end if
  end for
  $xk \leftarrow xHistory(k,:)$
  $[uk, nloptions, info] \leftarrow$ `nlmpcmove`$(nlobj, xk, lastMV, yref, [], nloptions$ ▷ Compute the optimal control moves with reference previewing
  $infoHistory(k,1) \leftarrow info.Iterations$ ▷ Store optimisation data for debugging
  $infoHistory(k,2) \leftarrow info.Cost$
  $infoHistory(k,3) \leftarrow info.ExitFlag$
  $uHistory(k + 1,:) \leftarrow uk'$ ▷ Store voltage data in voltage history matrix
  $ODEFUN \leftarrow nlStateFcn$ ▷ Set state function as the ODE to be solved
  $YOUT \leftarrow ode45(ODEFUN, [0\ Ts], ...$
$xHistory(k,:)')$ ▷ Solve ODE using `ode45` solver for init. conds. [`0 Ts`] across $xHistory$
  $xHistory(k + 1,:) \leftarrow YOUT(end,:)$ ▷ Save last row of (converged) states resulting from ODE solving
end for

Unfortunately, the simulation did not work for the sample time value suggested by Fig. 26 of `Ts = 0.01`. This is because the NLMPC controller has to have a prediction horizon of at least 2 seconds to provide optimal control inputs. For a sample time of 0.01, this would require a prediction horizon of `p = 200` time steps and therefore a control horizon of `c = 50`, a 10-fold increase (relative to Fig. 9) in a parameter that increases computational time exponentially.

When the simulation was run with a built-in timer, each time step took at least 8 hours, with at least 300 iterations in each time step. This iteration limit per time step was then reduced to 50, with the `EnableSuboptimalSolution` setting set to true. To help the optimisation system converge to a minimum within this iteration limit, initial guesses were supplied for the values of the inputs at each time step for the entirety of the prediction horizon. These MV guesses were pulled directly from the values obtained for the linear-model/linear-MPC simulation. Despite these efforts, each step took approximately two hours to complete. Even by reducing the duration from 10s to 5s, the 500 time steps would have taken 42 days to complete on a modern, dedicated i7-10700 processor running at 2.90 GHz. The sample time was thus increased to `Ts = 0.05`, for which an optimal solution was found in under 6 hours. Table 4 shows the rest of the variables that were used for this simulation, and Fig. 27 shows the quadcopter z position and rotor inputs, superimposed with the results from the linear MPC/linear model simulation (Fig. 9).

Table 4: Nonlinear $z = 1$ simulation parameters

Parameter	Variable	Value
Sample Time	`Ts`	0.05
Prediction Horizon	`p`	20
Control Horizon	`c`	5
MV Weight	`MVWeight`	0.1
x, y and z Weight	`OutputWeight`	10



(a) Z position (b) Rotor Inputs

Figure 27: NLMPC simulation for $z = 1$

Note that once again the voltages shown in Fig. 27b are pre-bias. The nominal voltage of 5.87V required to keep the quadcopter floating is added to these values to obtain the actual voltage required in practice. As can be seen from Fig. 27a, the NLMPC/nonlinear-model exhibits slightly worse performance the linear-MPC/linear-model. However, considering a vaster amount of quadcopter dynamics are captured by the nonlinear model, it is remarkable that the solver was able to compute an optimal control sequence for such a strongly nonlinear system.

5 Discussion

A robust MPC controller was designed during the research phase of this report. This novel controller has considerable safety implications as the amount of potential applications for drones in modern 21st century life grows exponentially. The multiple-MPC approach taken has not previously been explored to this extent. Efforts to use MPC as a basis for designing a robust closed-loop quadcopter controller have been limited to linear and hybrid MPC [36], and linear MPC with a feedback linearization loop [37] [38].

5.1 Errors and Uncertainties

Some sources of errors that could induce uncertainties in the research detailed in this report relate to the mathematical modelling of the quadcopter and the simulation parameters. Like all mathematical models, this one is simply an approximation of practical quadcopter dynamics. The performance of the controller has not been tested in areas such as sensor noise suppression, disturbance rejection, and sensitivity to model parameter uncertainty. These have all been dealt with in various ways [39] [40] [41]. Relative to those papers, the modelling and simulation of the controller in both the standard and failure operating conditions is closer to theory than practice. Rigorous experimental validation should be carried out prior to applying this controller to a quadcopter in a real-world situation.

5.2 Ethical Implications

The safety implications of a reliable, robust quadcopter controller are numerous. With drones picking up blood samples in India [42], delivering food in western countries [43], and locating wildfires in Australia and California [44], a quadcopter's ability to avoid crashing, maintain hover, and even complete its mission in the eventuality of rotor failure(s) is of incredible importance to the continued development of these systems.

5.3 Future Work

This is only one method of looking at robust quadcopter control. A recent paper has shifted the computational resources used to solve the robust quadcopter control from the processing unit (Multiple MPC state estimation) to an onboard sensing suite (event camera) [45]. As stated in the literature review section, research is being done into using MPC as a starting point for a more streamlined LP-based Model Predictive Controller [28].

Considering the rotor failure scenario, perhaps the LQR version could be modified to provide a stable system response by linearising the model across multiple operating conditions and using a gain-scheduling technique. A logic-based subsystem could be implemented that would detect if, and which, rotors are unresponsive, sending a signal back to the controller block which would correlate to a certain `K` matrix. Additional gain matrices could be designed and tuned for a larger flight envelope, such as for sharp lateral manoeuvres, a situation where LQR control proved unstable.

Wind disturbances of various sizes could also be added to the simulation environment, in order to evaluate each controller's response to a step gust input. However, LQR and MPC are both theorised to provide satisfactory performance in this area, this feature being therefore unlikely to change the outcomes of this report.

Finally, quadratic programming could be used to rewrite the optimisation and cost functions used by the NLMPC to compute the optimal control inputs, both of which could potentially lead to reductions in computational expense.

6 Conclusions

In this report, two methods for controlling a quadcopter were detailed and implemented into an existing quadcopter model, provided by Dr. Jonathan du Bois of the University of Bath. A literature review was initially carried out of different ways the quadcopter system has been modelled previously, the importance of fault detection and control, and commonly used control methods. The first control method to be researched was the Linear-Quadratic Regulator (LQR), which provides optimally controlled feedback instantaneously to the control system via a single matrix manipulation. It was found to be highly efficient for simple reference trajectories, especially in purely vertical flight. However, when horizontal displacement of more than 2 units was required, the system response would become unstable.

Conversely, the MPC control system optimised the inputs over a specified prediction horizon to minimise the state errors. This resulted in improved performance of $\approx 11\%$ at the cost of

computational time increase of 20.4%. The MPC system was able to utilise a wider range of the allowable voltage as its aggression was more easily tunable than that of the LQR.

Rotor failures were then implemented into the simulation. Rotor 1 was set to fail at $t = 11$s, and rotor 3 at $t = 21$s, with the simulation itself running until $t = 30$s. The LQR version and MPC version showed unstable behaviour, requiring a new kind of controller to be designed. A multiple-MPC subsystem was designed, where several controllers were connected in parallel to the reference signal and quadcopter model to provide a variety of optimal control signals. The estimated states from each of the 7 controllers were compared to the actual quadcopter's states, with the most accurate controller being switched to at each time step if the improvement over its predecessor crossed a certain threshold tolerance. This provided stable and accurate performance as opposed to the other control methods tested.

Finally, the nonlinear model was focused on for the last set of simulations. A state function, in essence a prediction model for the nonlinear version of the quadcopter, was written based off of the nonlinear Simulink model. A validation script was run, where inputs obtained from the nonlinear LQR model were inputted to the state function. Satisfactory performance was only obtained near sample time values of 0.01, which became the starting point for the actual NLMPC simulation. However, this sample time was found to be too computationally expensive, with time steps taking over two hours each, as the prediction and control horizon had to be increased by a factor of 10. The sample time was therefore increased to `Ts` $= 0.05$, which provided adequate performance, albeit slightly inferior to that of the linear MPC model. Also detailed were the multiple future research avenues that could be taken based on the research detailed in this report.

References

[1] Jonathan du Bois. "Cyclocopter Flight Dynamics Simulation and Control". *University of Bath* (2021).

[2] Gary J Balas. "Linear, parameter-varying control and its application to aerospace systems". *ICAS congress proceedings*. 2002. URL: `http://www.icas.org/ICAS_ARCHIVE/ICAS2002/PAPERS/541.PDF`.

[3] Wilson J Rugh and Jeff S Shamma. "Research on gain scheduling". *Automatica* 36.10 (2000), pp. 1401–1425.

[4] Veronica Adetola and Martin Guay. "Robust adaptive MPC for constrained uncertain nonlinear systems". *International Journal of Adaptive Control and Signal Processing* 25.2 (2011), pp. 155–167.

[5] Grzegorz Szafranski and Roman Czyba. "Different approaches of PID control UAV type quadrotor" (2011).

[6] Xingling Shao, Jun Liu, and Honglun Wang. "Robust back-stepping output feedback trajectory tracking for quadrotors via extended state observer and sigmoid tracking differentiator". *Mechanical Systems and Signal Processing* 104 (2018), pp. 631–647.

[7] Andrew Zulu and Samuel John. "A review of control algorithms for autonomous quadrotors". *arXiv preprint arXiv:1602.02622* (2016).

[8] Elias Reyes-Valeria et al. "LQR control for a quadrotor using unit quaternions: Modeling and simulation". *CONIELECOMP 2013, 23rd International Conference on Electronics, Communications and Computing*. IEEE. 2013, pp. 172–178.

[9] Iman Sadeghzadeh et al. "Fault-tolerant trajectory tracking control of a quadrotor helicopter using gain-scheduled PID and model reference adaptive control". *Annual Conference of the PHM Society*. Vol. 3. 1. 2011.

[10] Mahyar Abdolhosseini, Youmin M Zhang, and Camille Alain Rabbath. "An efficient model predictive control scheme for an unmanned quadrotor helicopter". *Journal of intelligent & robotic systems* 70.1 (2013), pp. 27–38.

[11] Youmin Zhang and Jin Jiang. "Bibliographical review on reconfigurable fault-tolerant control systems". *Annual reviews in control* 32.2 (2008), pp. 229–252.

[12] N. Eva Wu et al. "Sensor fault masking of a ship propulsion system". *Control Engineering Practice* 14.11 (2006), pp. 1337–1345. ISSN: 0967-0661. DOI: https://doi.org/10.1016/j.conengprac.2005.09.003.

[13] Paul M Frank and Birgit Köppen-Seliger. "Fuzzy logic and neural network applications to fault diagnosis". *International journal of approximate reasoning* 16.1 (1997), pp. 67–88.

[14] Y.M. Zhang et al. "Development of advanced FDD and FTC techniques with application to an unmanned quadrotor helicopter testbed". *Journal of the Franklin Institute* 350.9 (2013), pp. 2396–2422. ISSN: 0016-0032. DOI: https://doi.org/10.1016/j.jfranklin.2013.01.009.

[15] FE Thau. "Observing the state of non-linear dynamic systems". *International journal of control* 17.3 (1973), pp. 471–479.

[16] A Freddi, S Longhi, and A Monteriu. "A model-based fault diagnosis system for a mini-quadrotor". *7th workshop on Advanced Control and Diagnosis*. 2009, pp. 19–20.

[17] MH Moradi. "New techniques for PID controller design". *Proceedings of 2003 IEEE Conference on Control Applications, 2003. CCA 2003.* Vol. 2. IEEE. 2003, pp. 903–908.

[18] Su Whan Sung and In-Beum Lee. "Limitations and countermeasures of PID controllers". *Industrial & engineering chemistry research* 35.8 (1996), pp. 2596–2610.

[19] Anmol Agarwal, Ee Meng Ng, and Kin Huat Low. "Adaptive Control of Unmanned Quadrotor with Partial Actuator Failure using Model Reference Adaptive Control (MRAC) with Dynamic Inversion". *2021 International Conference on Unmanned Aircraft Systems (ICUAS)*. 2021, pp. 10–19. DOI: 10.1109/ICUAS51884.2021.9476830.

[20] Damiano Rotondo, Fatiha Nejjari, and Vicenç Puig. "Robust quasi-LPV model reference FTC of a quadrotor UAV subject to actuator faults". *International Journal of Applied Mathematics and Computer Science* 25.1 (2015).

[21] Alexander Lanzon, Alessandro Freddi, and Sauro Longhi. "Flight control of a quadrotor vehicle subsequent to a rotor failure". *Journal of Guidance, Control, and Dynamics* 37.2 (2014), pp. 580–591.

[22] Daewon Lee, H Jin Kim, and Shankar Sastry. "Feedback linearization vs. adaptive sliding mode control for a quadrotor helicopter". *International Journal of control, Automation and systems* 7.3 (2009), pp. 419–428.

[23] Sihao Sun et al. "Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors". *IEEE Transactions on Robotics* 37.1 (2020), pp. 116–130.

[24] Lucas M. Argentim et al. "PID, LQR and LQR-PID on a quadcopter platform". *2013 International Conference on Informatics, Electronics and Vision (ICIEV)*. 2013, pp. 1–6. DOI: 10.1109/ICIEV.2013.6572698.

[25] Kyaw Myat Thu and A.I. Gavrilov. "Designing and Modeling of Quadcopter Control System Using L1 Adaptive Control". *Procedia Computer Science* 103 (2017). XII International Symposium Intelligent Systems 2016, INTELS 2016, 5-7 October 2016, Moscow, Russia, pp. 528–535. ISSN: 1877-0509. DOI: https://doi.org/10.1016/j.procs.2017.01.046.

[26] Frank Allgöwer and Alex Zheng. *Nonlinear model predictive control*. Vol. 26. Birkhäuser, 2012.

[27] Rolf Findeisen, Frank Allgöwer, and Lorenz T Biegler. *Assessment and future directions of nonlinear model predictive control*. Vol. 358. 7. Springer, 2007.

[28] Sergey Vichik and Francesco Borrelli. "Solving linear and quadratic programs with an analog circuit". *Computers & Chemical Engineering* 70 (2014), pp. 160–171.

[29] Simon Newman. *Foundations of helicopter flight*. Elsevier, 1994.

[30] M Drela. "XFOIL: an analysis and design system for low Reynolds number analysis". *Springer Notes in Engineering* 54 (1989), pp. 1267–1289.

[31] Jérémy Jimenez et al. "Induced velocity model in steep descent and vortex-ring state prediction". *27th European Rotorcraft Forum* ().

[32] David A Peters and Shyi-Yaung Chen. "Momentum Theory, Dynamic Inflow, and the Vortex-Ring State". *Journal of the American Helicopter Society* 27.3 (1982), pp. 18–24.

[33] Brian L Stevens, Frank L Lewis, and Eric N Johnson. *Aircraft control and simulation: dynamics, controls design, and autonomous systems*. John Wiley & Sons, 2015.

[34] David W King, Allen Bertapelle, and Chad Moses. "UAV failure rate criteria for equivalent level of safety". *International helicopter safety symposium*. Vol. 8. Citeseer. 2005.

[35] S. Jamal Haddadi, P. Zarafshan, and M. Dehghani. "A coaxial quadrotor flying robot: Design, analysis and control implementation". *Aerospace Science and Technology* 120 (2022), p. 107260. ISSN: 1270-9638. DOI: https://doi.org/10.1016/j.ast.2021.107260.

[36] A. Bemporad, C.A. Pascucci, and C. Rocchi. "Hierarchical and Hybrid Model Predictive Control of Quadcopter Air Vehicles". *IFAC Proceedings Volumes* 42.17 (2009). 3rd IFAC Conference on Analysis and Design of Hybrid Systems, pp. 14–19. ISSN: 1474-6670. DOI: https://doi.org/10.3182/20090916-3-ES-3003.00004.

[37] Weihua Zhao and Tiauw Hiong Go. "Quadcopter formation flight control combining MPC and robust feedback linearization". *Journal of the Franklin Institute* 351.3 (2014), pp. 1335–1355. ISSN: 0016-0032. DOI: https://doi.org/10.1016/j.jfranklin.2013.10.021.

[38] Ngoc Thinh Nguyen, Ionela Prodan, and Laurent Lefevre. "Multi-layer optimization-based control design for quadcopter trajectory tracking". *2017 25th Mediterranean Conference on Control and Automation (MED)*. 2017, pp. 601–606. DOI: 10.1109/MED.2017.7984183.

[39] A Benallegue, A Mokhtari, and L Fridman. "Feedback linearization and high order sliding mode observer for a quadrotor UAV". *International Workshop on Variable Structure Systems, 2006. VSS'06*. IEEE. 2006, pp. 365–372.

[40] David Lara et al. "Parametric robust stability analysis for attitude control of a four-rotor mini-rotorcraft". *Proceedings of the 45th IEEE Conference on Decision and Control*. IEEE. 2006, pp. 4351–4356.

[41] Jun Li and Yuntang Li. "Dynamic analysis and PID control for a quadrotor". *2011 IEEE International Conference on Mechatronics and Automation*. IEEE. 2011, pp. 573–578.

[42] BBC. *What a drone picking up blood samples tells about healthcare in India*. 2022. URL: https://www.bbc.co.uk/news/world-asia-india-61267750 (visited on 05/02/2022).

[43] Isabel Cameron. *Delivery drones: dystopian or the future of ecommerce delivery?* 2022. URL: https://www.chargedretail.co.uk/2022/04/14/delivery-drones-dystopian-or-the-future-of-ecommerce-delivery/ (visited on 05/02/2022).

[44] Michelle Hampson. *Drones and Sensors Could Spot Fires Before They Go Wild*. 2021. URL: https://spectrum.ieee.org/drones-sensors-wildfire-detection (visited on 05/02/2022).

[45] Sihao Sun et al. "Autonomous quadrotor flight despite rotor failure with onboard vision sensors: Frames vs. events". *IEEE Robotics and Automation Letters* 6.2 (2021), pp. 580–587.

Appendix A: Linear state space matrix

A.1: Matrix A

	u	v	w	p	q	r	x	y	z	ϕ	θ	ψ	ω_1	ω_2	ω_3	ω_4
u	-1.0975e-06	0	0	0	0	0	0	0	0	0	-9.81	0	0	0	0	0
v	0	-1.0975e-06	0	0	0	0	0	0	0	9.81	0	0	0	0	0	0
w	0	0	-1.7105e-06	0	0	0	0	0	0	0	0	0	-0.0107	-0.0107	-0.0107	-0.0107
p	-4.4322e-05	0	0	0	0	0	0	0	0	0	0	0	-0.1067	0.1067	0.1067	-0.1067
q	0	-4.4322e-05	0	0	0	0	0	0	0	0	0	0	0.1067	0.1067	-0.1067	-0.1067
r	0	0	0	0	0	0	0	0	0	0	0	0	0.0034	-0.0034	0.0034	-0.0034
x	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
y	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
z	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
ϕ	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
θ	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
ψ	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
ω_1	0	0	-8.619e-05	0	0	0	0	0	0	0	0	0	-1.1063	0.0034	-0.0034	0.0034
ω_2	0	0	-8.619e-05	0	0	0	0	0	0	0	0	0	0.0034	-1.1063	0.0034	-0.0034
ω_3	0	0	-8.619e-05	0	0	0	0	0	0	0	0	0	-0.0034	0.0034	-1.1063	0.0034
ω_4	0	0	-8.619e-05	0	0	0	0	0	0	0	0	0	0.0034	-0.0034	0.0034	-1.1063

A.2: Matrix B

	V_1	V_2	V_3	V_4
u	0	0	0	0
v	0	0	0	0
w	0	0	0	0
p	0	0	0	0
q	0	0	0	0
r	-0.4459	0.4459	-0.4459	0.4459
x	0	0	0	0
y	0	0	0	0
z	0	0	0	0
ϕ	0	0	0	0
θ	0	0	0	0
ψ	0	0	0	0
ω_1	62.1542	-0.4459	0.4459	-0.4459
ω_2	-0.4459	62.1542	-0.4459	0.4459
ω_3	0.4459	-0.4459	62.1542	-0.4459
ω_4	-0.4459	0.4459	-0.4459	62.1542